ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN CHUYÊN NGÀNH

PHÁT TRIỂN GAME
FISHY BUSINESS

NGÀNH: KHOA HỌC MÁY TÍNH

HỘI ĐỒNG:	14L Đồ án chuyên ngành	
GVHD:	Th.S Vương Bá Thịnh	
SVTH1:	Lê Ngọc Hiền	2211024
SVTH2:	Huỳnh Trần Học Đăng	2210731
SVTH3:	Nguyễn Mạnh Hùng	2211337

Thành phố Hồ Chí Minh, tháng 12/2025

Mục lục

1 Giới thiệu **5**
1.1 Bối cảnh và vấn đề đặt ra . . . 5
1.2 Saboteur - Lựa chọn trong vô vàn board game . . . 5
1.3 Mục tiêu của đề tài . . . 10
1.4 Phạm vi thực hiện . . . 10
1.5 Ý nghĩa và tính ứng dụng của đề tài . . . 10
1.6 So sánh một số game tương tự trên thị trường . . . 11
1.6.1 Tabletop Simulator (PC - Steam) . . . 11
1.6.2 Board Game Arena (Web-based) . . . 11
1.6.3 Sự khác biệt của Fishy Business . . . 12

2 Yêu cầu của đề tài **13**
2.1 Yêu cầu chức năng . . . 13
2.1.1 Nhóm chức năng Gameplay (Core Loop - Saboteur) . . . 13
2.1.2 Nhóm chức năng Xã hội và Tương tác (Social & Interactive) . . . 14
2.2 Yêu cầu phi chức năng . . . 14
2.3 Yêu cầu kỹ thuật . . . 15
2.4 Yêu cầu trải nghiệm người dùng . . . 15

3 Nghiên cứu tổng quan **17**
3.1 Tổng quan về game 3D Multiplayer . . . 17
3.2 Nghiên cứu và lựa chọn mô hình phát triển phần mềm . . . 17
3.2.1 Mô hình Thác nước (Waterfall) . . . 18
3.2.2 Mô hình Agile . . . 19
3.2.3 Quy trình Scrum . . . 20
3.2.4 So sánh các mô hình phát triển dự án phần mềm . . . 20
3.3 Nghiên cứu và lựa chọn các công cụ hỗ trợ quản lý dự án . . . 21
3.3.1 Discord . . . 21
3.3.2 Git . . . 22
3.3.3 Github . . . 23
3.3.4 Jira . . . 23
3.4 Nghiên cứu so sánh và lựa chọn Game Engine . . . 24
3.4.1 Unity . . . 24
3.4.2 Unreal Engine . . . 25
3.4.3 Godot . . . 26
3.4.4 So sánh các game engine . . . 26

4 Cơ sở lý thuyết và công nghệ sử dụng cho networking **28**
4.1 So sánh các giải pháp Networking . . . 28
4.1.1 Photon . . . 28
4.1.2 Mirror Networking: . . . 30
4.1.3 Netcode for GameObjects (NGO) . . . 31
4.1.4 Netcode for Entities . . . 32
4.2 Lý do chọn Unity NGO . . . 32
4.3 Kiến trúc Network . . . 32
4.3.1 Mô hình Client-Host . . . 32
4.3.2 Mô hình Client-Server (Dedicated Server) . . . 33
4.3.3 Lý do chọn kiến trúc Client-Host ở giai đoạn đầu . . . 34
4.3.4 Lý do chuyển sang kiến trúc Client-Server ở giai đoạn sau . . . 35
4.4 Giải pháp voice chat Vivox . . . 35
4.5 Design Pattern trong dự án . . . 36
4.5.1 Singleton . . . 36
4.5.2 Observer Pattern . . . 37
4.5.3 State Pattern . . . 38

5 Thiết kế hệ thống **41**
5.1 Kiến trúc tổng thể . . . 41
5.1.1 Kiến trúc dự án Unity . . . 41
5.1.2 Kiến trúc NGO . . . 44
5.2 Hệ thống Lobby . . . 46
5.2.1 Tạo phòng . . . 47
5.2.2 Tham gia phòng . . . 48
5.2.3 Rời phòng . . . 48
5.2.4 Kick khỏi phòng . . . 48
5.2.5 Đồng bộ trạng thái phòng (Polling) . . . 48
5.2.6 Duy trì phòng bằng Heartbeat . . . 49
5.3 Hệ thống voice chat Vivox tích hợp với Lobby . . . 49
5.4 Thiết kế Gameplay . . . 49

6 Tiến độ dự án **51**

7 Kế hoạch thực hiện giai đoạn tiếp theo **60**
7.1 Kế hoạch theo mốc thời gian . . . 60
7.2 Mục tiêu và kết quả mong đợi . . . 60

8 Tài liệu tham khảo **62**

Danh sách hình vẽ

1 Bộ bài Saboteur đầy đủ . . . 6
2 Hình minh họa bộ thẻ . . . 7
3 Phân loại thẻ đường . . . 8
4 Mô hình Waterfall . . . 18
5 Mô hình Agile . . . 19
6 Scrum Process . . . 20
7 Tính năng chính của Photon Fusion . . . 29
8 Tính năng chính của Photon Quantum . . . 29
9 Gói dịch vụ miễn phí của Photon (PUN, Fusion, Quantum) cho bản phát triển . . . 30
10 Gói dịch vụ miễn phí của Photon (Fusion, Quantum) cho bản phát hành . . . 30
11 Gói dịch vụ trả phí của Photon (PUN, Fusion, Quantum) . . . 30
12 Gói dịch vụ cho Multiplayer của Unity . . . 31
13 Mô hình Client-Host . . . 33
14 Mô hình Client-Server (Dedicated Server) . . . 34
15 Cấu trúc mẫu của các GameObject trong Scene (GameScene) . . . 42
16 Cấu trúc mẫu của các Component trong GameObject (Player) . . . 43
17 Các scriptable object sử dụng để lưu dữ liệu Card trong game . . . 43
18 Các prefab lưu trữ GameObject của UI các screen trong game . . . 44
19 Figma: Bộ thiết kế card . . . 53
20 Game Screenshot: Nhân vật khi mới tham gia vào Cabin . . . 53
21 Game Screenshot: Người chơi nhảy từ gác xuống tầng 1 . . . 54
22 Game Screenshot: Người chơi đứng trước gương thay đổi trang phục . . . 54
23 Game Screenshot: Người chơi chơi đàn piano bằng bàn phím . . . 55
24 Game Screenshot: Menu trong game . . . 55
25 Game Screenshot: Quản lý lobby, có thể đổi tên phòng, kick người chơi,.. . . . 56
26 Game Screenshot: Tương tác với vật thể khi tới gần . . . 56
27 Game Screenshot: Người chơi quay đầu nhìn xung quanh . . . 57
28 Game Screenshot: Khung cảnh của 1 bàn chơi dưới góc nhìn mặc định . . . 57
29 Game Screenshot: Sử dụng thể rockfall (bomb) để phá đường . . . 58
30 Game Screenshot: Nối đường tới goal nhưng không chứa vàng . . . 58
31 Game Screenshot: Giao diện thắng/thua khi xong game . . . 59

Danh sách bảng

1 Bảng So sánh các mô hình phát triển dự án phần mềm . . . 21
2 So sánh ngắn gọn Unity, Unreal Engine và Godot . . . 27
3 So sánh 3 giải pháp Photon hỗ trợ . . . 28

4 Phân công nhiệm vụ giai đoạn 1 theo Sprint 52
5 Bảng kế hoạch thực hiện dự án giai đoạn 2 60

1 Giới thiệu

Mỗi trò chơi đều có bối cảnh ra đời, mục tiêu phát triển và phạm vi triển khai. Ngoài ra, không thể thiếu những điểm khác biệt so với các sản phẩm hiện có trên thị trường. Trong phần này, nhóm sẽ làm rõ những điều trên.

1.1 Bối cảnh và vấn đề đặt ra

Thị trường game hiện nay đang chứng kiến sự trỗi dậy mạnh mẽ của dòng game Social Deduction (Suy luận xã hội) và Party Game. Những cái tên như Among Us, Goose Goose Duck hay Party Animals đã chứng minh một chân lý: người chơi hiện đại không chỉ tìm kiếm thử thách về kỹ năng, họ khao khát sự kết nối. Họ muốn một không gian ảo nơi họ có thể tương tác, lừa lọc, cười đùa và "tương tàn" với bạn bè mình.

Bên cạnh đó, với công nghệ ngày càng phát triển ngày nay, các trò chơi điện tử từ đó cũng ngày càng được phổ biến với mọi người trên thế giới. Trong đó yếu tố 3D trong các trò chơi điện tử đã trở nên không quá xa lạ trên thế giới và ngày càng trở nên được yêu thích. Yếu tố 3D mang đến cho người chơi một gốc nhìn mới mẻ, rộng rãi và chân thực hơn, khả năng đem đến trải nghiệm tốt cho người chơi được cao hơn. Đồ họa 3D xuất hiện trong các trò chơi đã trở nên kích thích với không chỉ giới trẻ mà gần như với lứa tuổi trên thế giới.

Tuy nhiên, khi nhìn vào các bản chuyển thể Board Game (trò chơi bàn cờ) lên nền tảng số hiện nay, nhóm nhận thấy một "lỗ hổng" rất lớn. Hầu hết các game Saboteur (Kẻ phá hoại) trên thị trường đều chỉ tập trung vào cơ chế bài bạc (2D) trên một mặt phẳng tĩnh. Điều này vô tình giết chết không khí của một buổi tụ tập. Cảm giác căng thẳng khi nhìn vào mắt đối phương, hay sự vui vẻ khi cùng nhau làm trò hề trong lúc chờ đến lượt đi... hoàn toàn biến mất.

Từ đó, nhóm đã chọn đề tài phát triển một trò chơi dạng Party Game kết hợp với board game và các hoạt động ngoài lề khác như thay đồ, đánh đàn, ... Trò chơi kết hợp giữa board game, multiplayer và sự sáng tạo trong công nghệ trò chơi. Việc đưa các hoạt động ngoài lề vào trò chơi sẽ làm tăng thêm sự tương tác của các người chơi với nhau. Điều này có thể tạo nên sự mới lạ, thu hút người chơi khám phá các tính năng trong trò chơi, nhưng cũng cần bảo đảm giữ được các yếu tố cơ bản và tinh thần chiến thuật truyền thống của trò chơi chính là Saboteur để giúp người chơi không bị quá xa lạ với trò chơi.

1.2 Saboteur - Lựa chọn trong vô vàn board game

Board game Saboteur ra mắt năm 2004, được thiết kế bởi Fréderic Moyersoen, nhanh chóng trở thành một trong những trò chơi nhóm nổi tiếng nhất nhờ lối chơi bluffing - tương tác - phản bội cực kỳ kịch tính. Lấy bối cảnh những người thợ mỏ cùng nhau đào đường hầm tìm kho báu

nhưng luôn có kẻ phá hoại ẩn mình, Saboteur tạo nên sự pha trộn hoàn hảo giữa chiến thuật, đọc vị tâm lý và những màn phản đòn bất ngờ. Nhờ luật chơi đơn giản nhưng độ sâu chiến thuật cao, trò chơi phù hợp với nhiều độ tuổi và được dịch, phát hành rộng rãi trên toàn thế giới. Với tính chất tương tác mạnh và khả năng tạo ra vô số tình huống hài hước lẫn "plot twist", Saboteur hoàn toàn xứng đáng được tái hiện trong một trò chơi online 3D nhiều người chơi, nơi người chơi không chỉ chơi bài mà còn giao tiếp, hành động và trải nghiệm cảm giác nghi kỵ - hợp tác sống động hơn bao giờ hết trong không gian cabin ảo.

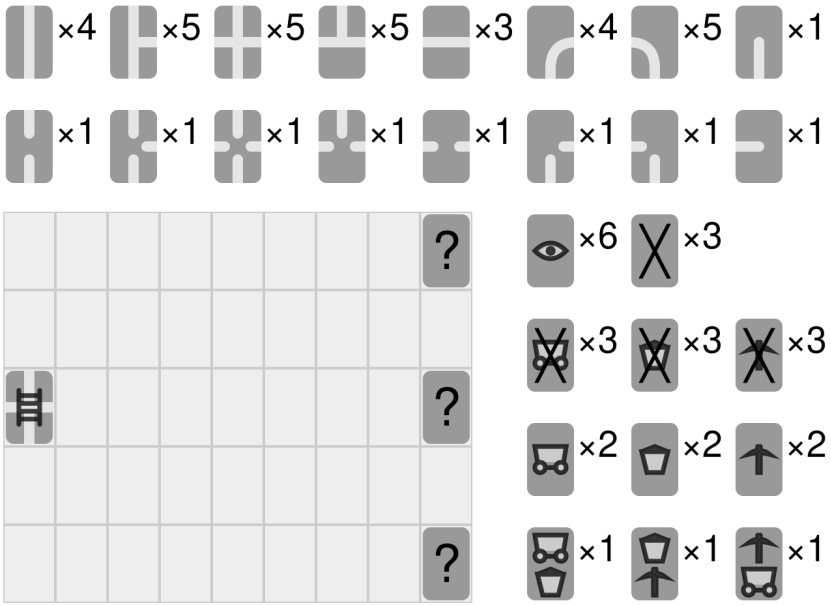


Hình 1: Bộ bài Saboteur đầy đủ

Luật chơi của Saboteur [1] cụ thể như sau:

1. **Tổng quan và mục tiêu**
   - Số người chơi: 3–10.
   - Mục tiêu:
     - **Gold-Diggers**: mở đường từ Start đến Goal chứa vàng.
     - **Saboteurs**: phá để Gold-Diggers không thể đến được vàng.
2. **Trang bị trong bộ Saboteur**
   - 40 Thẻ đường (path cards): gồm 31 thẻ thông, 9 thẻ cụt
   - 27 Thẻ hành động (action cards)
   - 11 Thẻ vai trò (role cards)

3. **Bố trí bàn chơi (Setup)**



Hình 2: Hình minh họa bộ thẻ

- Đặt thẻ Start ở giữa bên trái.
- Đặt 3 thẻ Goal úp cách Start 7 khoảng thẻ: 1 giữa, 1 trên, 1 dưới.
- Vai trò và số thẻ phát cho mỗi người
  Số lượng vai trò theo số người chơi:

Tổng số Người chơi	3	4	5	6	7	8	9	10
Số Saboteurs	1	1	2	2	3	3	3	4
Số Gold-Diggers	2	3	3	4	3	4	5	6

  Số thẻ phát cho mỗi người:

Tổng số Người chơi	3	4	5	6	7	8	9	10
Thẻ mỗi người	6	6	6	5	5	4	4	4

- Tất cả những lá bài còn lại được xáo lại thành chồng và để úp xuống bàn.

4. **Luật chơi**

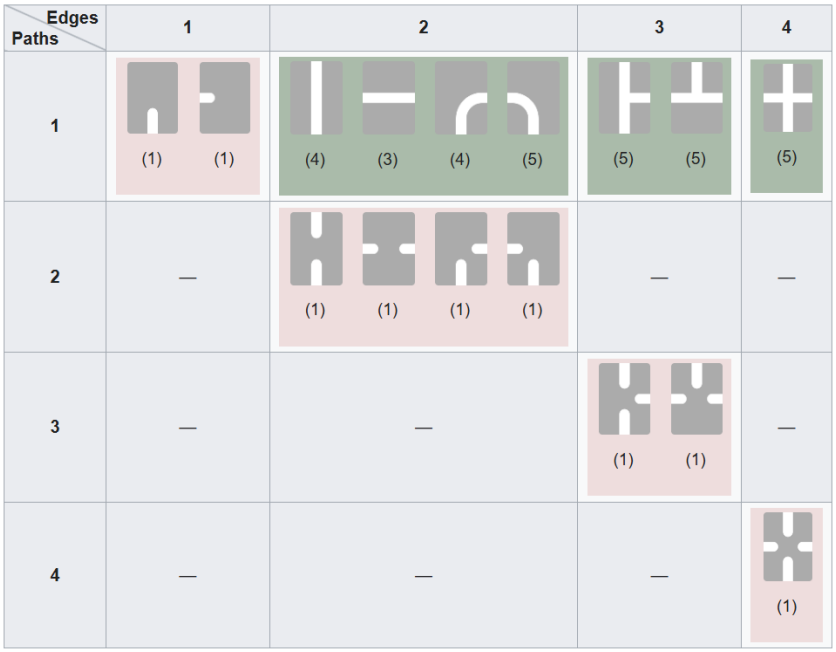
- Lượt xoay chiều kim đồng hồ.
- Mỗi lượt chọn một trong các hành động:
  (a) Đặt thẻ đường (path card).

(b) Chơi thẻ hành động (action card).

(c) Bỏ thẻ (discard).

- Sau khi xong lượt của mình, người chơi bốc lên thẻ trên cùng của deck, lượt chơi tiếp tục qua người tiếp theo.
- Gold-Diggers thắng khi có đường nối (phải là đường thông) từ start tới goal chứa vàng. Saboteurs thắng khi tất cả đã hết bài mà vẫn chưa nối được đường tới vàng.

5. **Thẻ đường (Path Cards)**



Hình 3: Phân loại thẻ đường

- **Phân loại thẻ Đường** Thẻ Đường (*Path cards*) trong Saboteur được dùng để tạo ra mạng lưới đường hầm từ vị trí xuất phát đến các thẻ mục tiêu. Chúng được chia thành:
  - **Thẻ đường thẳng**: nối liền hai đầu theo trục ngang hoặc dọc.
  - **Thẻ giao nhau**: có nhiều nhánh rẽ, cho phép mở rộng đường theo nhiều hướng.
  - **Thẻ cụt (dead-end)**: Có thể nối với các thể khác nhưng không được tính là thông đường.
  - **C**ác cạnh thẻ không có lối được tính là tường, khiến người chơi không thể nối đường theo hướng đó.
- **Cách đặt thẻ Đường hợp lệ** Một thẻ Đường được xem là hợp lệ khi thỏa các điều kiện sau:

  - Các đầu đường (lối ra) trên thẻ mới phải khớp với đầu đường của thẻ đã có trên bàn: đường nối với đường, tường nối với tường.
  - Không được tạo ra một kết nối trái ngược (ví dụ: đầu đường mở nối vào tường bịt).
  - Thẻ phải được đặt kề cạnh một thẻ đã có sẵn trong mạng lưới (không được "nhảy cóc").
- **Quy tắc xoay thẻ** Tất cả thẻ Đường đều có thể được xoay 180° trước khi đặt xuống.
  - Người chơi không được xoay 90°; chỉ xoay 180° vì cấu trúc hầm được thiết kế theo hai hướng chính.
  - Sau khi xoay, thẻ vẫn phải thỏa điều kiện kết nối hợp lệ như đã nêu ở mục trên.
- **Phá đường bằng thẻ Rockfall** Thẻ *Rockfall* là thẻ hành động dùng để phá hủy một thẻ Đường đã được đặt trên bàn.
  - Người chơi chọn một thẻ Đường bất kỳ (trừ thẻ Start và thẻ Goal) để loại bỏ.
  - Sau khi phá, ô trống được để lại và không có thẻ nào tự động dịch chuyển vào.
  - Rockfall không được dùng để phá thẻ trên tay người chơi, chỉ phá thẻ đã đặt.

6. **Thẻ hành động (Action Cards)**

   Các thẻ hành động ảnh hưởng trực tiếp đến khả năng đặt thẻ đường hoặc quan sát thông tin trong ván chơi. Trong game có **3 loại công cụ**:

   - Xe đẩy (*Cart*)
   - Xẻng (*Shovel*)
   - Mũ (*Hat*)

   Nếu người chơi bị phá **bất kỳ một trong ba công cụ**, họ **không thể đặt thẻ đường**. Để sửa, người chơi phải dùng thẻ Repair tương ứng.

   - **Repair đơn**: sửa đúng 1 công cụ được chỉ định trên thẻ.
   - **Repair kép**: sửa 1 trong 2 công cụ hiển thị trên thẻ (Có biến thể có thể sửa cả hai đồng thời).

Loại thẻ	Công dụng	Số lượng
Sabotage	Phá 1 công cụ: Cart, Shovel hoặc Hat. Người bị phá không thể đặt Path cho đến khi được sửa.	9
Repair	Sửa công cụ bị phá. Gồm: 6 thẻ đơn (mỗi loại công cụ 2 thẻ), 3 thẻ kép.	9
Map	Cho phép xem lén 1 trong 3 thẻ Goal. Không được tiết lộ thông tin trừ khi tự nguyện.	6
Rockfall	Phá hủy 1 Path card trên bàn (trừ Start và 3 Goal). Thẻ bị phá được loại khỏi mỏ.	3

1.3 Mục tiêu của đề tài

Từ những phân tích về thực trạng và nhu cầu thị trường nêu trên, nhóm thực hiện đề tài xác định các mục tiêu cốt lõi cần đạt được như sau:

Thứ nhất, số hóa và tự động hóa trọn vẹn luật chơi Saboteur: Mục tiêu đầu tiên là xây dựng một hệ thống logic game hoàn chỉnh, đảm bảo tuân thủ chính xác luật chơi gốc của board game Saboteur. Hệ thống phải tự động xử lý các thao tác phức tạp như chia bài, kiểm tra vai trò (Role), thuật toán loang để xác định đường đi liên thông từ điểm xuất phát đến kho báu, và tính toán thắng thua. Điều này nhằm loại bỏ sự rườm rà trong việc thiết lập bàn chơi (setup) thủ công, giúp người chơi tập trung hoàn toàn vào chiến thuật.

Thứ hai, xây dựng môi trường tương tác xã hội 3D: Khác với các ứng dụng chơi bài 2D truyền thống, mục tiêu của nhóm là tạo ra một không gian 3D sống động – nơi người chơi không chỉ là những cái tên trên bảng điểm. Người chơi sẽ được hóa thân vào các nhân vật (Avatar), có khả năng di chuyển tự do trong không gian sảnh chờ (Lobby) và bàn chơi. Mục tiêu là tái hiện cảm giác "tụ tập" thực tế, nơi các tương tác vật lý và phi ngôn ngữ được đề cao.

Thứ ba, phát triển các tính năng giải trí bổ trợ: Để giải quyết bài toán "thời gian chết" (downtime) khi chờ đợi người khác đánh bài hoặc chờ đủ người tạo phòng, nhóm đặt mục tiêu tích hợp các mini-features mang tính giải trí cao. Cụ thể là hệ thống nhạc cụ ảo (cho phép người chơi tương tác phím để tạo âm thanh) và hệ thống thời trang (thay đổi ngoại hình nhân vật). Đây là yếu tố then chốt để giữ chân người chơi và tạo nên sự vui nhộn đặc trưng của dòng Party Game.

Thứ tư, hoàn thiện kỹ thuật Multiplayer thời gian thực: Ứng dụng kiến thức về lập trình mạng để xây dựng hệ thống Client-Host ổn định. Đảm bảo sự đồng bộ mượt mà về vị trí nhân vật, trạng thái bàn chơi và các tương tác thời gian thực giữa nhiều người chơi khác nhau, giảm thiểu độ trễ (latency) để trải nghiệm không bị gián đoạn.

1.4 Phạm vi thực hiện

Về nền tảng, trò chơi hướng đến việc chơi trò chơi trên nền tảng Window từ phiên bản Window 7 trở lên.

Về đối tượng hướng đến của trò chơi, trò chơi hướng đến tất cả mọi người ở mọi lứa tuổi cần sử giải trí, sự mới mẻ, sự cạnh tranh và tư duy giải đố.

1.5 Ý nghĩa và tính ứng dụng của đề tài

Ý nghĩa khoa học và thực tiễn: Việc thực hiện đề tài "Fishy Business" không chỉ là việc tạo ra một sản phẩm giải trí đơn thuần, mà còn là cơ hội để nhóm áp dụng tổng hợp các kiến thức nền tảng của ngành Khoa học máy tính.

- *Thử thách về Lập trình mạng:* Đồng bộ hóa trạng thái trong môi trường 3D là một bài toán khó, đòi hỏi sự hiểu biết sâu sắc về cấu trúc gói tin, băng thông và xử lý độ trễ.
- *Tư duy Thiết kế hệ thống (System Design):* Việc kết hợp giữa logic chặt chẽ của một game bài (Turn-based) với sự tự do của một game nhập vai (Real-time 3D) đòi hỏi một kiến trúc phần mềm linh hoạt, dễ bảo trì và mở rộng (Design Patterns).

Tính ứng dụng:

- *Giải trí và Kết nối:* Trong bối cảnh làm việc và học tập từ xa ngày càng phổ biến, sản phẩm đóng vai trò là cầu nối giúp bạn bè, đồng nghiệp có không gian để "gặp gỡ" và giải trí cùng nhau mà không bị giới hạn bởi khoảng cách địa lý.
- *Tiềm năng mở rộng:* Dự án được xây dựng với kiến trúc hướng đối tượng (OOP), do đó có thể dễ dàng mở rộng để tích hợp thêm nhiều board game khác nhau (như Ma Sói, Mèo Nổ) vào cùng một "vũ trụ" sảnh chờ 3D này trong tương lai, hướng tới mô hình "Metaverse" thu nhỏ cho board game.

1.6 So sánh một số game tương tự trên thị trường

Để định vị sản phẩm rõ ràng hơn, nhóm tiến hành so sánh *Fishy Business* với các sản phẩm cạnh tranh trên nền tảng PC và Web.

1.6.1 Tabletop Simulator (PC - Steam)

Đây là "tượng đài" của dòng game mô phỏng board game trên PC.

- **Điểm mạnh:** Mô phỏng vật lý cực kỳ chân thực, kho game khổng lồ từ Workshop.
- **Điểm yếu cần khắc phục:** Tabletop Simulator là một môi trường "Sandbox" thuần túy. Nó không có luật chơi tự động (người chơi phải tự tính điểm, tự xào bài bằng tay ảo). Hơn nữa, nó khá nặng nề và phức tạp cho những người chơi casual (phổ thông). Người chơi không có nhân vật đại diện (Avatar) để đi lại tương tác, dẫn đến thiếu cảm giác kết nối xã hội.

1.6.2 Board Game Arena (Web-based)

Nền tảng chơi board game trên trình duyệt web rất phổ biến.

- **Điểm mạnh:** Nhanh, tiện, chạy trên mọi trình duyệt, luật chơi tự động hóa tốt.
- **Điểm yếu cần khắc phục:** Đồ họa 2D hoàn toàn tĩnh. Trải nghiệm chơi giống như đang làm việc với một file Excel có hình ảnh hơn là chơi game. Không có âm nhạc, không có không khí, không có tương tác giữa các người chơi ngoài khung chat text bé tí.

1.6.3 Sự khác biệt của Fishy Business

Fishy Business chọn một lối đi riêng trên PC:

- **Trải nghiệm 3D Native:** Không phải là web game, đây là một ứng dụng Windows Standalone tận dụng sức mạnh đồ họa để tạo ra không gian đẹp mắt.
- **Kết hợp hoàn hảo:** Nhóm lấy sự tiện lợi (luật chơi tự động) của *Board Game Arena* kết hợp với sự tự do, vui nhộn (di chuyển, nghịch ngợm) của các game như *Overcooked* hay *Party Animals*.
- **Tính năng độc nhất:** Khả năng sử dụng bàn phím PC để chơi nhạc cụ (Jamming) là tính năng "Signature" (đặc trưng) mà các đối thủ khác không có, biến sảnh chờ thành một sân khấu thực thụ.

2 Yêu cầu của đề tài

Dựa trên mục tiêu xây dựng một tựa game Multiplayer kết hợp giữa tư duy chiến thuật và tương tác xã hội trên nền tảng PC, nhóm xác định các yêu cầu cụ thể đối với hệ thống như sau. Việc phân định rõ ràng các nhóm yêu cầu này là kim chỉ nam cho quá trình thiết kế và lập trình ở các giai đoạn sau.

2.1 Yêu cầu chức năng

Đây là những tính năng cốt lõi mà hệ thống bắt buộc phải thực hiện được để đáp ứng nhu cầu của người chơi. Nhóm chia thành hai phân hệ chính: Gameplay và Social.

2.1.1 Nhóm chức năng Gameplay (Core Loop - Saboteur)

Hệ thống phải đảm bảo vận hành chính xác logic của board game Saboteur gốc:

- **Quản lý phiên chơi:**
  - Cho phép người chơi Tạo phòng (Host Game) và Tham gia phòng (Join Game) thông qua mã phòng hoặc danh sách phòng.
  - Hỗ trợ tối thiểu 2 người và tối đa 8 người trong một phiên chơi.
- **Cơ chế chia bài và Phân vai:**
  - Hệ thống tự động xáo bài và chia bài ngẫu nhiên cho người chơi theo số lượng quy định.
  - Phân vai ngẫu nhiên (Thợ mỏ hoặc Kẻ phá hoại) và đảm bảo bí mật vai trò của từng người chơi (chỉ người chơi đó mới thấy vai trò của mình).
- **Cơ chế Đánh bài:**
  - *Thẻ đường hầm (Path Cards):* Người chơi có thể đặt thẻ lên bàn chơi. Hệ thống phải kiểm tra tính hợp lệ (logic khớp nối đường đi) trước khi cho phép đặt.
  - *Thẻ chức năng (Action Cards):* Hỗ trợ các thẻ phá hủy đường hầm, xem bản đồ kho báu, hoặc khóa/mở khóa hành động của người chơi khác.
- **Cơ chế Chiến thắng:**
  - Tự động phát hiện khi đường hầm nối liền từ điểm xuất phát đến kho báu (Phe Thợ mỏ thắng).
  - Tự động kết thúc ván khi hết bài mà chưa tìm thấy kho báu (Phe Phá hoại thắng).

2.1.2 Nhóm chức năng Xã hội và Tương tác (Social & Interactive)

Tận dụng lợi thế 3D và nền tảng PC để tăng cường trải nghiệm:

- **Điều khiển nhân vật:** Người chơi điều khiển nhân vật 3D di chuyển tự do trong sảnh chờ bằng cụm phím WASD và điều hướng camera bằng Chuột.
- **Hệ thống Nhạc cụ:**
  - Cung cấp các nhạc cụ ảo (Guitar, Piano, ...) trong môi trường game.
  - Cho phép người chơi kích hoạt nhạc cụ và sử dụng bàn phím máy tính để chơi các nốt nhạc (Note mapping). Âm thanh phát ra phải được đồng bộ đến tai của tất cả người chơi khác trong phòng.
- **Hệ thống Thời trang:**
  - Người chơi có thể truy cập kho đồ để thay đổi trang phục (Skin, Mũ, Phụ kiện) cho nhân vật.
  - Sự thay đổi ngoại hình phải được cập nhật ngay lập tức cho các người chơi khác (kể cả late-joiners) cùng thấy.
- **Giao tiếp:** Hỗ trợ kênh Chat văn bản (Global Chat), hệ thống biểu cảm (Emotes/Animations) như vẫy tay, nhảy múa và voice chat.

2.2 Yêu cầu phi chức năng

Yêu cầu về chất lượng và hiệu suất hoạt động của hệ thống:

- **Hiệu năng:**
  - Game phải duy trì tốc độ khung hình ổn định (tối thiểu 60 FPS) trên các cấu hình máy tính tầm trung (có card đồ họa rời phổ thông hoặc onboard đời mới).
  - Thời gian tải màn chơi không quá 10 giây.
- **Độ trễ mạng:**
  - Các thao tác quan trọng như đánh bài, di chuyển nhân vật phải được đồng bộ với độ trễ thấp (dưới 100ms trong điều kiện mạng ổn định) để tránh hiện tượng "teleport" hoặc lệch pha.
  - Đặc biệt, tính năng đánh đàn yêu cầu độ trễ cực thấp để đảm bảo nhịp điệu bài hát không bị vỡ khi nhiều người cùng chơi (Jamming).
- **Tính tin cậy:**
  - Hệ thống phải xử lý được các ngoại lệ: người chơi đột ngột mất kết nối không được làm sập phòng chơi của những người còn lại.

  - Cơ chế Re-connect cơ bản nếu người chơi bị rớt mạng trong thời gian ngắn.

- **Khả năng mở rộng:** Kiến trúc code phải được tổ chức theo hướng Modular (mô-đun hóa) để dễ dàng thêm các loại bài mới, role mới hoặc nhạc cụ mới trong tương lai mà không phá vỡ logic cũ.

2.3 Yêu cầu kỹ thuật

Các ràng buộc về công nghệ và môi trường phát triển:

- **Nền tảng vận hành:**
  - Hệ điều hành: Windows 10 (64-bit) hoặc Windows 11.
  - Thiết bị ngoại vi: Bắt buộc sử dụng Chuột và Bàn phím.
- **Công cụ phát triển:**
  - *Game Engine:* Ổn định, hỗ trợ lập trình 3D, multiplayer và hỗ trợ build PC mạnh mẽ.
  - *IDE:* Visual Studio 2022 hoặc JetBrains Rider.
- **Giải pháp mạng:**
  - Sử dụng kiến trúc Client-Server hoặc Host-Client.
  - Thư viện mạng: Netcode for GameObjects (NGO) hoặc Photon Fusion (tùy chọn dựa trên độ phức tạp).
- **Tài nguyên đồ họa (Assets):**
  - Phong cách nghệ thuật: Low-poly và Cel-shading để phù hợp với không khí vui nhộn và tối ưu hiệu năng.

2.4 Yêu cầu trải nghiệm người dùng

Đối với một game Party, cảm xúc của người chơi là yếu tố sống còn. Nhóm đặt ra các tiêu chuẩn UX sau:

- **Tính trực qua:**
  - Giao diện người dùng (UI) phải rõ ràng. Các lá bài trên tay phải dễ nhìn, dễ đọc thông tin.
  - Thao tác kéo thả bài hoặc click chọn bài phải mượt mà, có phản hồi bằng âm thanh hoặc hiệu ứng hình ảnh khi tương tác thành công.
- **Bầu không khí:**

  - Môi trường 3D phải tạo cảm giác ấm cúng, vui vẻ.
  - Âm nhạc nền nhẹ nhàng, không lấn át tiếng trò chuyện hoặc tiếng nhạc cụ do người chơi tạo ra.

- **Tính phản hồi:**
  - Các thông báo quan trọng (tới lượt chơi, tìm thấy vàng, bị phá bài) phải hiển thị nổi bật giữa màn hình.
  - UI phản hồi nhanh chóng khi nhận tương tác.
  - Khi người chơi bấm phím đánh đàn, âm thanh phải phát ra ngay lập tức để tạo cảm giác thỏa mãn.

- **Khả năng tiếp cận:** Game cung cấp hướng dẫn cơ bản hoặc Tooltip giải thích công dụng của từng lá bài khi di chuột vào, giúp người chưa từng chơi Saboteur cũng có thể nắm bắt nhanh chóng.

3 Nghiên cứu tổng quan

Để xây dựng thành công tựa game "Fishy Business", nhóm đã tiến hành nghiên cứu các cơ sở lý thuyết và công nghệ nền tảng liên quan đến phát triển trò chơi điện tử trực tuyến. Nội dung nghiên cứu bao gồm tổng quan về game multiplayer, quy trình phát triển phần mềm, so sánh các game engine phổ biến và kiến trúc mạng máy tính.

3.1 Tổng quan về game 3D Multiplayer

Trò chơi nhiều người chơi (Multiplayer Game) là một hệ thống thời gian thực phức tạp, nơi nhiều người chơi tương tác với nhau trong cùng một môi trường ảo.

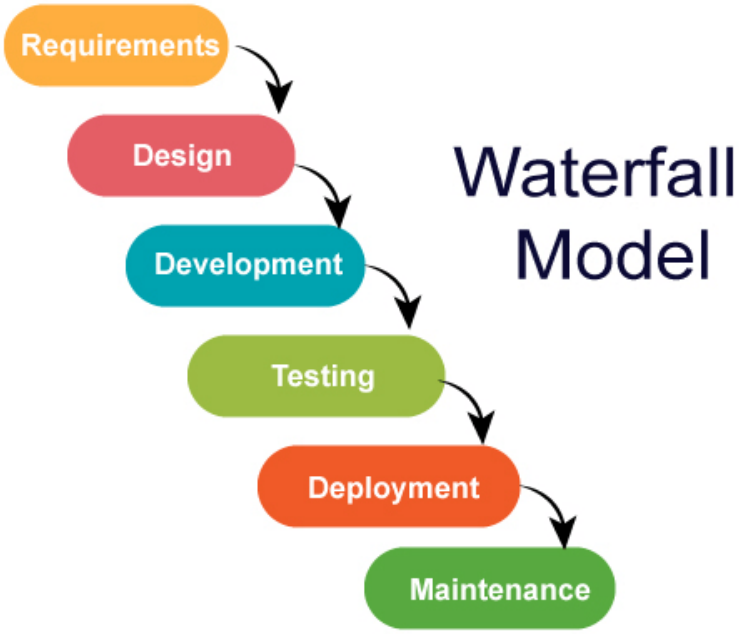
Đặc điểm cốt lõi: Khác với game Single-player (nơi mọi logic diễn ra cục bộ trên một máy), game Multiplayer đối mặt với thách thức lớn nhất là sự **đồng bộ hóa trạng thái**. Mọi hành động của người chơi A (di chuyển, đánh bài, đánh đàn) phải được truyền tải chính xác và gần như tức thời đến người chơi B, C, D,...

- *Không gian 3D:* Việc đồng bộ trong không gian 3D phức tạp hơn 2D do phải xử lý dữ liệu biến đổi liên tục của vector vị trí (Position), góc xoay (Rotation) và các trạng thái vật lý (Physics) trên cả 3 trục (x, y, z).
- *Vấn đề độ trễ:* Đây là yếu tố sống còn. Trên môi trường Internet, gói tin luôn có độ trễ. Các kỹ thuật như *Client-side Prediction* (Dự đoán phía máy khách) và *Lag Compensation* (Bù trễ) là những chủ đề nghiên cứu bắt buộc để đảm bảo trải nghiệm mượt mà.

3.2 Nghiên cứu và lựa chọn mô hình phát triển phần mềm

Việc lựa chọn quy trình phát triển phần mềm phù hợp quyết định đến khả năng quản lý rủi ro và tiến độ của dự án. Nhóm đã nghiên cứu một số mô hình phổ biến sau:

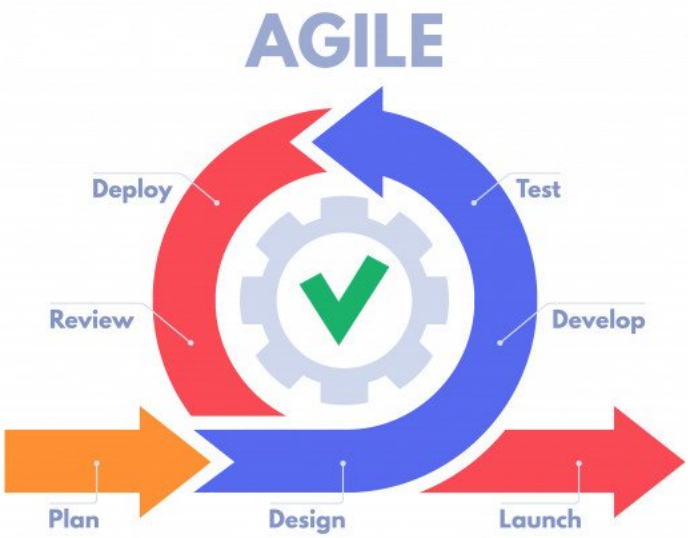
3.2.1 Mô hình Thác nước (Waterfall)



Hình 4: Mô hình Waterfall

Waterfall model được coi là mô hình phát triển phần mềm đầu tiên được sử dụng. Lý do việc mô hình này được gọi là thác nước vì các giai đoạn phát triển được thực hiện tuần tự và nối tiếp nhau, như một dòng nước chảy. Mỗi giai đoạn hoàn thành trước khi giai đoạn tiếp theo bắt đầu và không có sự quay lại. Do tính chất này, mỗi giai đoạn của mô hình thác nước phải được xác định chính xác.

3.2.2 Mô hình Agile

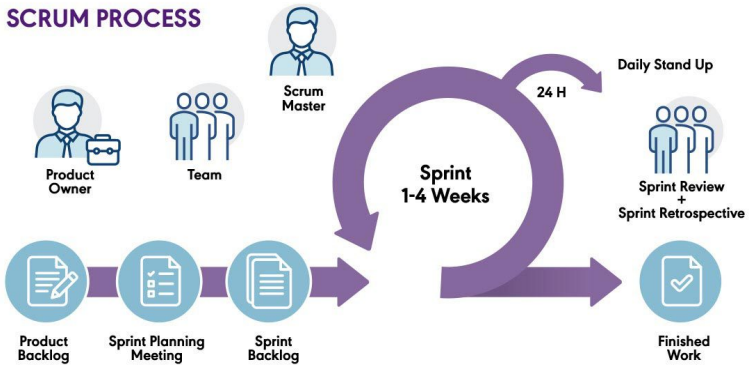


Hình 5: Mô hình Agile

Mô hình Agile là một phương pháp phát triển phần mềm linh hoạt. Agile model được tạo ra dựa trên 2 mô hình: Iterative (Lặp lại) và Incremental (Tăng dần). Nó tập trung vào việc phân phối sản phẩm thông qua các chu kỳ phát triển ngắn (gọi là sprint) và thường xuyên nhận phản hồi từ khách hàng.

Agile giúp nhóm phát triển dễ dàng thích nghi với các thay đổi yêu cầu, đồng thời cải thiện chất lượng sản phẩm liên tục.

3.2.3 Quy trình Scrum



Hình 6: Scrum Process

Scrum là một "khung quản lý dự án" được áp dụng rất rộng rãi từ những dự án đơn giản với một nhóm phát triển nhỏ cho đến những dự án có yêu cầu rất phức tạp với hàng trăm người tham gia. Ngoài ra, Scrum Process cũng phù hợp với những dự án đòi hỏi khung thời gian cố định.

Trong Scrum, công việc sẽ được chia nhỏ thành nhiều giai đoạn gọi là Sprint. Mỗi Sprint chỉ kéo dài từ 1 đến 4 tuần, không quá một tháng. Đầu Sprint sẽ lên kế hoạch làm những yêu cầu nào rồi thực hiện code và test. Cuối Sprint là một sản phẩm hoàn thiện cả code lẫn test có thể demo và chạy được.

3.2.4 So sánh các mô hình phát triển dự án phần mềm

Mỗi mô hình phát triển dự án đều có các đặc điểm nổi trội riêng, Do đó để phân tích rõ hơn việc lựa chọn giữa các mô hình phát triển để thực hiện dự án, dưới đây là bảng so sánh điểm mạnh và điểm yếu giữa các mô hình:

Mô hình	Điểm mạnh	Điểm yếu
Waterfall	• Phù hợp với dự án nhỏ và ngắn hạn • Dễ quản lý, có kế hoạch rõ ràng từ đầu. • Có sự ổn định cao, ít thay đổi và xác định yêu cầu rõ ràng.	• Rất khó để quay lại giai đoạn nào khi nó đã kết thúc • Ít tính linh hoạt và phạm vi điều chỉnh của nó khá là khó khăn, tốn kém.
Agile	• Tăng cường khả năng làm việc nhóm. • Các chức năng được rõ ràng và dễ quản lý. • Có tính linh hoạt cao.	• Không phù hợp với các dự án phức tạp. • Phụ thuộc vào yếu tố bên ngoài • Khó kiểm soát tiến độ và deadline. • Dễ mất định hướng nếu không có kế hoạch rõ ràng.
Scrum	• Quản lý tiến độ chặt chẽ theo Sprint. • Theo dõi hiệu suất làm việc rõ ràng. • Phân chia công việc cụ thể, rõ trách nhiệm.	• Nhiều họp, tốn thời gian quy trình. • Không linh hoạt như Agile thuần. • Hạn chế việc thay đổi đột ngột giữa Sprint.

Bảng 1: Bảng So sánh các mô hình phát triển dự án phần mềm

Qua bảng phân tích trên, nhóm nhận thấy các đặc điểm của mô hình Scrum là phù hợp với dự án của nhóm nhất. Do đó nhóm quyết định sử dụng mô hình Scrum để phát triển dự án.

3.3 Nghiên cứu và lựa chọn các công cụ hỗ trợ quản lý dự án

Trong quy trình Scrum, việc quản lý mã nguồn và tiến độ là bắt buộc. Do đó nhóm đã chọn các công cụ phổ biến sau.

3.3.1 Discord



Discord là một nền tảng giao tiếp trực tuyến được sử dụng rộng rãi trong lĩnh vực công nghệ thông tin nói chung và lập trình game nói riêng. Trong quá trình thực hiện dự án, Discord đóng vai trò là công cụ hỗ trợ chính cho việc quản lý nhóm, trao đổi kỹ thuật và phối hợp công việc giữa các thành viên. Nhóm sử dụng Discord để tổ chức các buổi họp trực tuyến, thảo luận về hướng phát triển game cũng như xử lý các vấn đề phát sinh trong quá trình làm.

Discord cho phép tạo nhiều kênh thảo luận khác nhau theo từng mục đích cụ thể như trao đổi chung, lập trình, thiết kế gameplay và báo lỗi. Việc phân chia kênh giúp nội dung trao đổi được sắp xếp khoa học, tránh chồng chéo thông tin và nâng cao hiệu quả làm việc nhóm. Bên cạnh đó, Discord còn hỗ trợ chia sẻ nhanh các tài nguyên như mã nguồn, hình ảnh, video minh họa gameplay và các tài liệu thiết kế.

Ngoài ra, nền tảng này còn hỗ trợ lưu trữ lịch sử trò chuyện, giúp các thành viên dễ dàng theo dõi lại quá trình thảo luận và quyết định kỹ thuật đã được thống nhất trước đó. Nhờ các tính năng trên, Discord góp phần nâng cao khả năng phối hợp, giảm thời gian trao đổi và hỗ trợ hiệu quả cho việc quản lý và phát triển dự án theo mô hình Scrum.

3.3.2 Git



Git là một hệ thống quản lý phiên bản phân tán mã nguồn mở, được Linus Torvalds phát triển vào năm 2005 nhằm đáp ứng nhu cầu quản lý mã nguồn hiệu quả trong quá trình phát triển nhân hệ điều hành Linux. Cho đến nay, Git đã trở thành một trong những hệ thống quản lý phiên bản phổ biến và được sử dụng rộng rãi nhất trong lĩnh vực phát triển phần mềm. Git cho phép theo dõi toàn bộ lịch sử thay đổi của mã nguồn và hỗ trợ nhiều người làm việc đồng thời trên cùng một dự án.

Một trong những ưu điểm nổi bật của Git là tốc độ xử lý nhanh, khả năng hoạt động ổn định ngay cả với các dự án có quy mô lớn. Git có giao diện dòng lệnh đơn giản, giúp các thành viên trong nhóm dễ dàng tiếp cận và sử dụng. Với đặc tính phân tán hoàn toàn, mỗi thành viên đều sở hữu một bản sao đầy đủ của mã nguồn trên máy cục bộ, cho phép làm việc ngoại tuyến và đồng bộ dữ liệu linh hoạt với các thành viên khác trong nhóm.

Ngoài ra, Git hỗ trợ mạnh mẽ cho việc phát triển song song thông qua cơ chế nhánh (branch), giúp nhiều người có thể phát triển các chức năng khác nhau cùng lúc mà không ảnh hưởng trực tiếp đến mã nguồn chính. Git còn sử dụng các hàm băm mật mã để đảm bảo tính toàn vẹn của dữ liệu, giúp phát hiện nhanh chóng các thay đổi không hợp lệ hoặc sai sót trong quá trình chỉnh sửa mã nguồn. Nhờ những ưu điểm trên, Git đóng vai trò quan trọng trong việc quản lý mã nguồn hiệu quả cho dự án của nhóm.

3.3.3 Github



GitHub là một nền tảng lưu trữ và quản lý mã nguồn trực tuyến dựa trên hệ thống quản lý phiên bản Git, được sử dụng rộng rãi trong cộng đồng phát triển phần mềm. Trong quá trình thực hiện dự án, GitHub được nhóm sử dụng để lưu trữ mã nguồn, đồng bộ dữ liệu giữa các thành viên và quản lý tiến độ phát triển dự án.

GitHub hỗ trợ làm việc nhóm thông qua các chức năng như kho lưu trữ (repository), nhánh (branch), gộp mã (merge) và yêu cầu hợp nhất (pull request). Nhờ đó, các thành viên có thể phát triển song song các chức năng khác nhau mà không gây ảnh hưởng trực tiếp đến mã nguồn chính. Ngoài ra, GitHub còn cung cấp hệ thống theo dõi lỗi (Issues), giúp ghi lại các lỗi phát sinh và phân công nhiệm vụ cho từng thành viên một cách rõ ràng.

Bên cạnh đó, GitHub còn lưu trữ đầy đủ lịch sử thay đổi của mã nguồn, giúp dễ dàng kiểm tra, khôi phục các phiên bản trước khi cần thiết. Nhờ những tính năng trên, GitHub đóng vai trò quan trọng trong việc quản lý mã nguồn, hỗ trợ làm việc nhóm hiệu quả và đảm bảo tính an toàn cho dự án của nhóm.

3.3.4 Jira



Jira là một công cụ quản lý dự án và theo dõi tiến độ công việc do công ty Atlassian phát triển, được sử dụng phổ biến trong các dự án phát triển phần mềm theo mô hình Agile và Scrum. Trong quá trình thực hiện dự án của nhóm, Jira được nhóm sử dụng để quản lý công việc, theo

dõi tiến độ từng giai đoạn và kiểm soát hiệu quả làm việc của các thành viên.

Jira cho phép tạo và quản lý các đầu việc (Issue) như Task, Bug, Story, từ đó phân công nhiệm vụ cụ thể cho từng thành viên trong nhóm. Mỗi công việc đều có trạng thái rõ ràng như "To Do", "In Progress", "Done", giúp nhóm dễ dàng theo dõi tiến độ thực hiện. Bên cạnh đó, Jira còn hỗ trợ xây dựng bảng Scrum, lập kế hoạch Sprint và theo dõi khối lượng công việc trong từng Sprint một cách trực quan.

Ngoài ra, Jira còn cung cấp các công cụ thống kê và báo cáo tiến độ như biểu đồ Burndown Chart, Velocity Chart, giúp nhóm đánh giá hiệu suất làm việc theo từng giai đoạn. Nhờ những tính năng trên, Jira hỗ trợ hiệu quả cho việc quản lý tiến độ, phân công nhiệm vụ và kiểm soát chất lượng dự án theo mô hình Scrum.

3.4 Nghiên cứu so sánh và lựa chọn Game Engine

Game engine là một phần mềm chuyên dụng được sử dụng để thiết kế và phát triển trò chơi điện tử. Có thể hiểu đơn giản, game engine đóng vai trò là một phần mềm trung gian, kết nối và quản lý sự tương tác giữa nhiều thành phần trong cùng một hệ thống trò chơi. Các chức năng cốt lõi của game engine chủ yếu liên quan đến việc làm việc trực tiếp với phần cứng, bao gồm: công cụ dựng hình đồ họa hai chiều và ba chiều, hệ thống vật lý dùng để phát hiện và xử lý va chạm, xử lý âm thanh, quản lý hình ảnh động.

Ngoài các chức năng cơ bản, nhiều game engine hiện nay còn tích hợp các gói mở rộng nhằm hỗ trợ những tính năng nâng cao như kết nối mạng, xử lý luồng dữ liệu, tối ưu hóa quá trình kết xuất hình ảnh và hiệu năng của trò chơi. Nhờ có game engine, quá trình phát triển trò chơi điện tử được rút ngắn đáng kể, giúp lập trình viên tiết kiệm thời gian và công sức thông qua việc tái sử dụng và mở rộng các thành phần có sẵn để tạo ra nhiều sản phẩm game khác nhau.

Hiện nay, trên thị trường có rất nhiều game engine với những ưu điểm và hạn chế riêng, phù hợp với từng thể loại trò chơi và mục đích phát triển khác nhau. Một số game engine phổ biến sẽ được trình bày ở các mục tiếp theo.

3.4.1 Unity



Unity là một game engine được phát hành lần đầu vào năm 2005 và nhanh chóng trở thành một trong những nền tảng phát triển trò chơi phổ biến nhất hiện nay. Unity được nhiều nhà phát triển lựa chọn nhờ tính linh hoạt, dễ sử dụng và khả năng hỗ trợ xây dựng trò chơi đa nền tảng như Windows, Linux, macOS, thiết bị di động (Android, iOS) và trình duyệt web. Game engine này đặc biệt phổ biến trong lĩnh vực phát triển game di động và các trò chơi độc lập (indie).

Unity hỗ trợ phát triển cả trò chơi hai chiều (2D) và ba chiều (3D), cho phép mô phỏng các tương tác và hiệu ứng trong môi trường ảo một cách trực quan. Ngôn ngữ lập trình chính được sử dụng trong Unity là C#, giúp lập trình viên dễ dàng tiếp cận và phát triển các chức năng cho trò chơi. Bên cạnh đó, Unity còn hỗ trợ các pipeline kết xuất hình ảnh hiện đại như HDRP (High Definition Render Pipeline) và URP (Universal Render Pipeline), giúp nâng cao chất lượng đồ họa và tối ưu hiệu năng.

Ngoài các công cụ phát triển cốt lõi, Unity còn cung cấp kho tài nguyên phong phú thông qua Unity Asset Store, bao gồm mô hình 3D, âm thanh, plugin và các công cụ hỗ trợ phát triển game. Đồng thời, Unity sở hữu một cộng đồng phát triển lớn và năng động, sẵn sàng hỗ trợ và giải đáp các thắc mắc cho người mới. Nhờ những ưu điểm trên, Unity là một lựa chọn phù hợp cho việc phát triển đồ án lập trình game của nhóm.

3.4.2 Unreal Engine



Unreal Engine là một game engine mạnh mẽ do Epic Games phát triển, được phát hành lần đầu vào năm 1998 và hiện nay đã trở thành một trong những công cụ phổ biến nhất trong lĩnh vực phát triển game 3D. Unreal Engine được sử dụng rộng rãi để phát triển các trò chơi có đồ họa chất lượng cao trên nhiều nền tảng như Windows, macOS, Linux, console và thiết bị di động.

Unreal Engine hỗ trợ lập trình bằng ngôn ngữ C++ và hệ thống lập trình trực quan Blueprint, giúp cả lập trình viên chuyên nghiệp lẫn người mới đều có thể tiếp cận và phát triển trò chơi. Engine này nổi bật với khả năng kết xuất hình ảnh chân thực, hệ thống ánh sáng tiên tiến, vật

lý mạnh mẽ và các công cụ hỗ trợ trí tuệ nhân tạo cho nhân vật trong game.

Ngoài ra, Unreal Engine còn cung cấp nhiều công cụ hỗ trợ phát triển như hệ thống vật lý, âm thanh, mạng và tối ưu hiệu năng. Cùng với cộng đồng người dùng lớn và kho tài nguyên phong phú, Unreal Engine là lựa chọn phù hợp cho các dự án game quy mô lớn và các trò chơi đòi hỏi chất lượng đồ họa cao.

3.4.3 Godot



Godot là một game engine mã nguồn mở được phát triển nhằm phục vụ cho việc xây dựng các trò chơi 2D và 3D. Godot nổi bật với ưu điểm miễn phí hoàn toàn, không yêu cầu trả phí bản quyền và cho phép người dùng toàn quyền chỉnh sửa mã nguồn theo nhu cầu. Nhờ đó, Godot được sử dụng rộng rãi trong cộng đồng lập trình game độc lập (indie) và giáo dục.

Godot hỗ trợ đa nền tảng như Windows, Linux, macOS, Android, iOS và trình duyệt web. Engine này sử dụng các ngôn ngữ lập trình như GDScript (ngôn ngữ riêng của Godot có cú pháp tương tự Python), C#, và C++. Hệ thống scene và node của Godot giúp việc tổ chức đối tượng trong game trở nên rõ ràng, linh hoạt và dễ quản lý.

Ngoài ra, Godot còn tích hợp sẵn các hệ thống vật lý, âm thanh, xử lý va chạm, hoạt ảnh và trí tuệ nhân tạo. Với dung lượng nhẹ, giao diện thân thiện, cộng đồng phát triển ngày càng lớn và tài liệu hướng dẫn phong phú, Godot là một lựa chọn phù hợp cho các dự án game nhỏ, game học tập và đồ án lập trình game.

3.4.4 So sánh các game engine

Mỗi phần mềm Game engine đều có các đặc điểm nổi trội riêng, Do đó để phân tích rõ hơn việc lựa chọn giữa các Game engine để thực hiện dự án, dưới đây là bảng so sánh điểm mạnh và điểm yếu giữa các phần mềm Game engine:

Tiêu chí	Unity	Unreal Engine	Godot
Ngôn ngữ	C#: Dễ học, phổ biến	C++ / Blueprint: Mạnh, khó	GDScript: Giống Python, dễ
Đồ họa	Tốt (URP, HDRP)	Rất cao (Nanite, Lumen)	Khá, nhẹ
2D/3D	Mạnh cả 2D và 3D	Mạnh 3D	Mạnh 2D, 3D cơ bản
Hiệu năng	Tốt, ổn định	Rất cao	Nhẹ, đủ dùng
Cộng đồng	Rất lớn	Lớn	Đang phát triển
Phần cứng	Trung bình	Cao	Thấp
Networking	Mạnh (Netcode, Photon)	Tích hợp tốt	Cơ bản
Chi phí	Miễn phí cơ bản	Có phí theo điều kiện	Miễn phí 100%
Phù hợp	Mobile, Indie, AR/VR	AAA, đồ họa cao	Học tập, Indie nhỏ

Bảng 2: So sánh ngắn gọn Unity, Unreal Engine và Godot

Kết luận: Nhóm chọn **Unity** vì sự cân bằng giữa hiệu năng và tính dễ sử dụng của ngôn ngữ C#, đồng thời hệ sinh thái hỗ trợ Multiplayer của Unity rất phong phú phù hợp cho quy mô nhóm sinh viên.

4 Cơ sở lý thuyết và công nghệ sử dụng cho networking

4.1 So sánh các giải pháp Networking

Hiện nay, trên thị trường có nhiều công cụ (framework) hỗ trợ lập trình multiplayer cho Unity rất mạnh mẽ. Trong đó có những công cụ bên thứ ba có thể tích hợp vào Unity như **Photon** (PUN, Fusion, Quantum), **Mirror Networking** và cũng có những công cụ do chính Unity phát triển như **Netcode for GameObjects** (NGO), **Netcode for Entities**. Sau đây, nhóm sẽ phân tích và so sánh các giải pháp trên nhằm đưa ra được lựa chọn hợp lý cho công cụ lập trình Networking của dự án.

4.1.1 Photon

Trong Photon có 3 giải pháp chính cho Networking là PUN (Photon Unity Networking), Fusion và Quantum. Dưới đây là so sánh tổng quan về 3 giải pháp trên.

Đặc điểm	PUN	Fusion	Quantum
Cơ chế đồng bộ	State sync (thủ công cổ điển)	Snapshot-based (Host mode/Dedicated Server)	Deterministic lockstep
Độ khó sử dụng	Dễ	Trung bình	Khó
Độ trễ	Trung bình	Thấp	Cực thấp
Ưu điểm	Dễ dùng, nhiều tài liệu hướng dẫn	Hiệu năng tốt hơn PUN, hỗ trợ rollback, prediction	Hiệu năng cao nhất, chống gian lận, đa nền tảng
Nhược điểm	Chậm, đã ngừng phát triển	Cần nhiều kiến thức networking hơn	Phải code ESC theo kiểu của Quantum
Dòng game phù hợp	Casual, board game (nhẹ, ít vật thể)	FPS, 3D action (yêu cầu realtime, nhiều tác động vật lý)	MOBA, Fighting (cần độ chính xác rất cao, anti-cheat)

Bảng 3: So sánh 3 giải pháp Photon hỗ trợ

Hiện tại, các giải pháp trên của Photon đều cung cấp gói miễn phí với giới hạn số người dùng trực truyến cùng lúc (CCU) tối đa là 20 và chỉ dành cho phiên bản phát triển (development only).

Riêng công cụ Fusion và Quantum thì có thêm 1 gói miễn phí cho phiên bản phát hành (chỉ 1 app duy nhất) với giới hạn CCU tối đa lên đến 100 kèm theo tăng traffic mỗi tháng (từ 60GB lên 0.3TB).

Các gói dịch vụ cao cấp hơn của Photon (mở giới hạn traffic, CCU) phải trả phí (95$/12 tháng, 125$/tháng, 250$/tháng, 500$/tháng)

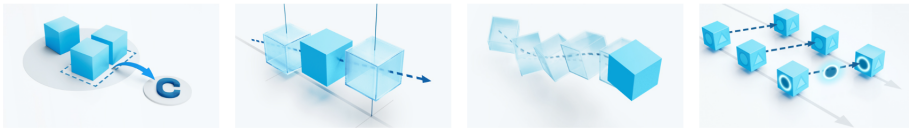


Tick-Based Simulation

Core feature of stable and accurate networking. Foundation for client side prediction and snapshot interpolation.

Client-Side Prediction

Give players an instant response to their own inputs without giving up server authority, even under high latency and network loss.



High Performance

Handle thousands of networked object over hundreds of client connections, including complex interest management.

Snapshot Interpolation

Provide smooth visual rendering of remote objects for a client, even during bad networking conditions.

Lag Compensation

Compcompensate for the delay between the client firing his weapon and the server registering the potential hit.

Replication Systems

Per object full consistency or eventual consistency.

Hình 7: Tính năng chính của Photon Fusion



No Netcode – Multiplayer by Default

With Quantum, you can create a multiplayer game without writing any netcode. Just create a single simulation with multiple connected players, similar to developing a local multiplayer experience.

Low Bandwidth With Input Only

Game clients only exchange input, distributed to the players at high frequencies through our Photon Cloud. Use webhooks to connect your backend or plugins to run server-side custom code.

Delivers Extreme Performance

Each component of Quantum's ECS architecture provides the necessary performance to run even physics-intensive multiplayer games on PCs, consoles, VR and on mobile phones.

Cheat Protection by Design

Deterministic games inherently resist cheating. Combat cheaters through methods like replay validation or server referee simulations which is the most effective anti-cheat measure.



Cross Platform Determinism

Simulations coded in Quantum do not have any dependencies on Unity and can run anywhere.

Zero Lag E-Sport Grade

All local actions are executed with zero lag while remote inputs are predicted and rolled-back.

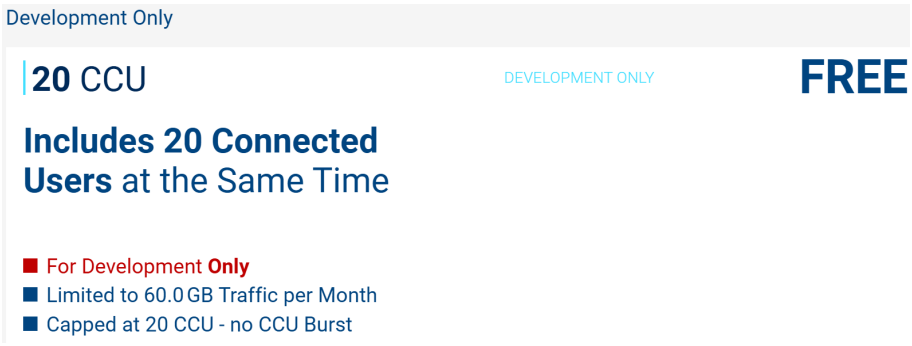
Replays & Spectating

Replays can be stored on your backend or used in-game as e.g. kill cams. Spectating is build-in.

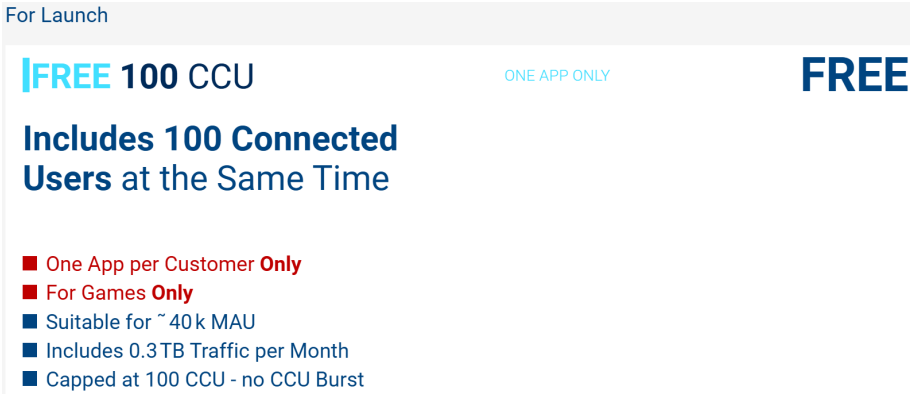
Bot SDK Add-On

Bots significantly impact multiplayer games by filling rooms and replacing departing players.

Hình 8: Tính năng chính của Photon Quantum



Hình 9: Gói dịch vụ miễn phí của Photon (PUN, Fusion, Quantum) cho bản phát triển



Hình 10: Gói dịch vụ miễn phí của Photon (Fusion, Quantum) cho bản phát hành



Hình 11: Gói dịch vụ trả phí của Photon (PUN, Fusion, Quantum)

4.1.2 Mirror Networking:

Mirror Networking là một thư viện mã nguồn mở dùng cho networking trong Unity. Thư viện này hoàn toàn miễn phí và luôn được cập nhật đều đặn đến hiện tại. Mirror Networking hỗ trợ host mode (P2P) và cả dedicated server. Nhờ những đặc tính của một thư viện mã nguồn mở, công cụ này cung cấp khả năng linh hoạt, tùy biến cao. Điều này phù hợp cho nhóm phát triển nhỏ (indie), yêu cầu cao cho việc kiểm soát logic server, mạng chặt chẽ, mở rộng tùy ý. Giải pháp

này không giới hạn về CCU, nhưng sẽ phụ thuộc vào cấu server mà nhà phát triển xây dựng.

4.1.3 Netcode for GameObjects (NGO)

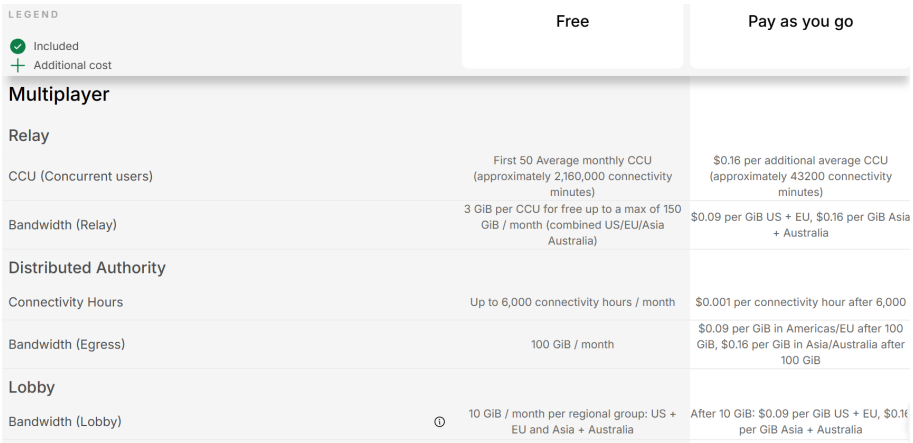
NGO (Netcode for GameObjects) là một công cụ chính thức được Unity phát triển và là giải pháp mặc định cho quá trình phát triển game Multiplayer trong unity. NGO có giao diện sử dụng dễ dàng, thân thuộc nhờ việc xây dựng trên workflow truyền thống của Unity (GameObject, MonoBehaviour,...).

Các dịch vụ Multiplayer khác của Unity cũng được tích hợp vào NGO:

- Lobby: có thể hỗ trợ lên đến 150 người chơi 1 phòng
- Player Authentication: xác thực người dùng đa nền tảng (facebook, google, anonymous,...)
- Matchmaker: hỗ trợ cơ chế ghép trận
- Relay: hỗ trợ cho P2P
- Friends; hỗ trợ cơ chế bạn bè
- Vivox: hỗ trợ cơ chế voice chat, text chat

NGO hỗ trợ 2 kiến trúc mạng chính là Client-Server (Dedicated server) và Client-Host. Về giao thức truyền tải mạng, NGO sử dụng UPD đạng tin cậy (Reliable UDP), giao thức này vừa kế thừa đặc điểm tốc độ cao của UDP truyền thống, vừa kết hợp đặc tính tin cậy (tính đảm bảo chuyển giao, tính thứ tự) của giao thức TCP.

Với gói dịch vụ miễn phí, NGO cung cấp giới hạn CCU tối đa mỗi tháng lên đến 50 và với mỗi mốc vượt ngưỡng sẽ trả phí (pay as you go).



LEGEND Included Additional cost	Free	Pay as you go
Multiplayer		
Relay		
CCU (Concurrent users)	First 50 Average monthly CCU (approximately 2,160,000 connectivity minutes)	$0.16 per additional average CCU (approximately 43200 connectivity minutes)
Bandwidth (Relay)	3 GiB per CCU for free up to a max of 150 GiB / month (combined US/EU/Asia Australia)	$0.09 per GiB US + EU, $0.16 per GiB Asia + Australia
Distributed Authority		
Connectivity Hours	Up to 6,000 connectivity hours / month	$0.001 per connectivity hour after 6,000
Bandwidth (Egress)	100 GiB / month	$0.09 per GiB in Americas/EU after 100 GiB, $0.16 per GiB in Asia/Australia after 100 GiB
Lobby		
Bandwidth (Lobby)	10 GiB / month per regional group: US + EU and Asia + Australia	After 10 GiB: $0.09 per GiB US + EU, $0.16 per GiB Asia + Australia

Hình 12: Gói dịch vụ cho Multiplayer của Unity

Tuy hỗ trợ nhiều dịch vụ, nhưng hệ thống của NGO chỉ phù hợp cho dự án quy mô nhỏ (khoảng 8-16 người chơi 1 phòng). Với các game có số lượng người chơi lớn, hiệu suất của NGO sẽ giảm mạnh. Ngoài ra, NGO cũng không hỗ trợ Client-side Prediction và Lag Compensation, điều này gây khó khăn trong việc hiện thực các game hành động nhanh, FPS khi nhà phát triển phải tự triển khai toàn bộ, làm tăng độ phức tạp của dự án.

4.1.4 Netcode for Entities

Tương tự NGO, Netcode for Entities cũng là một công cụ chính thức do Unity phát triển. Tuy nhiên, công cụ này tập trung tối ưu cho những game đòi hỏi số lượng người chơi lớn, xử lý nhiều logic, vật lý cho nhiều vật thể cùng lúc. Netcode for Entities được dùng kết hợp với hệ thống ECS (Entity Component System) và DOTS (Data-Oriented Technology Stack) để tối ưu hiệu suất xử lý đồng thời nhiều đối tượng. Ngoài ra, công cụ này cũng có thể dễ dàng tích hợp với dịch vụ của Unity như NGO.

4.2 Lý do chọn Unity NGO

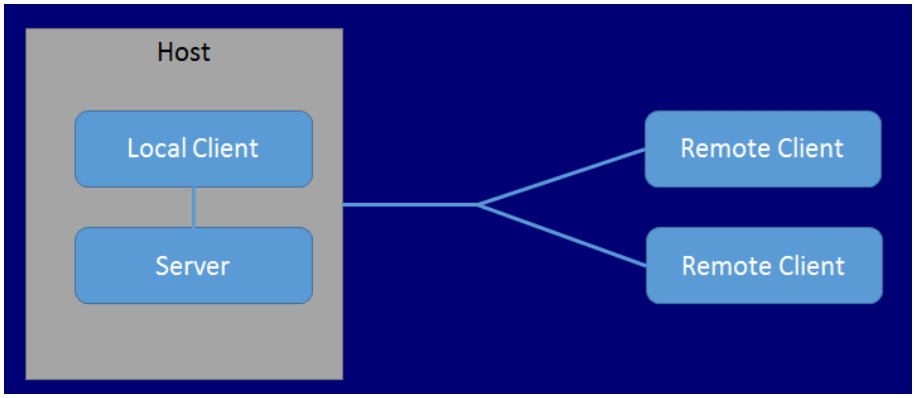
NGO được chọn vì là giải pháp chính thức và khuyến nghị từ Unity, đảm bảo tương thích cao và cập nhật, hỗ trợ thường xuyên. Nó hỗ trợ dễ dàng client-host model, tích hợp với Unity Multiplayer Services (như Relay cho NAT punchthrough), và hiệu suất ổn cho game 3D turn-based. So với Photon, NGO rẻ hơn (dựa trên Unity subscription); so với Mirror, NGO đơn giản hơn cho người mới. So với Netcode for Entities thì NGO đơn giản và phù hợp với dự án hơn, vì kích thước game giới hạn tối đa chỉ là 8 người chơi. Do vậy, NGO là giải pháp tốt nhất về độ phức tạp và chi phí, có thể triển khai dễ dàng cùng như thuận lợi cho việc phát triển các dịch vụ khác của Unity (authentication, voice chat,...).

4.3 Kiến trúc Network

Unity Netcode for GameObjects (NGO) cung cấp nhiều mô hình kiến trúc mạng khác nhau nhằm đáp ứng được nhu cầu của từng loại game multiplayer cụ thể. Mô hình kiến trúc mạng sẽ quyết định cách thức mà dữ liệu được truyền đi giữa các máy, việc máy nào sẽ thực thi logic, cũng như tính bảo mật và hiệu suất của hệ thống. NGO sử dụng 2 kiến trúc mạng chính là Client-Server (dedicated server) và Client-Host. Để lựa chọn được kiến trúc mạng phù hợp theo từng giai đoạn của dự án, sau đây nhóm sẽ phân tích tổng quan về hai kiến trúc mạng trên.

4.3.1 Mô hình Client-Host

Trong mô hình Client-Host, có một người chơi sẽ lá Host (vừa là Client, vừa là Server) chạy logic Client như những người chơi khác, đồng thời cũng chạy logic của Server. Các người chơi khác là Client sẽ kết nối tới người chơi Host như một kết nối tới Server.



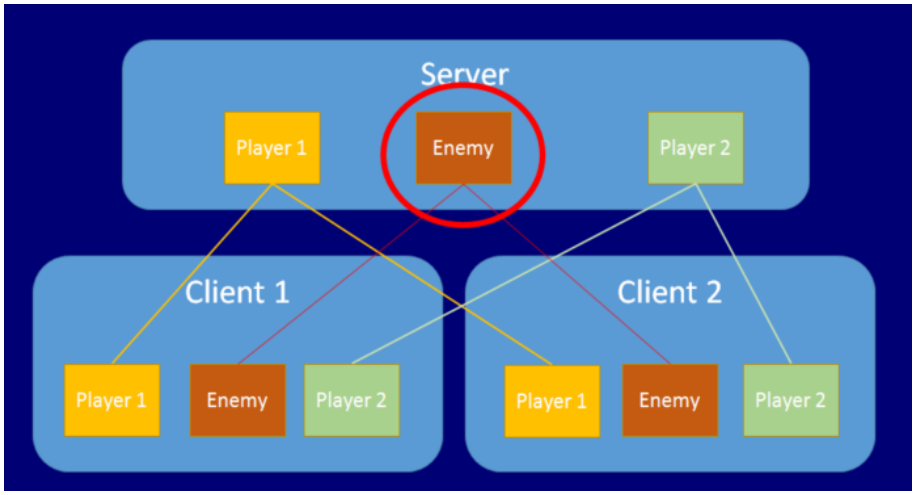
Hình 13: Mô hình Client-Host

- **Đặc điểm chính:**
  - + Server ở đây chạy trực tiếp trên máy người chơi (Host).
  - + Không dùng máy chủ riêng.
  - + Máy người chơi Host xử lý toàn bộ logic gameplay và đồng bộ các trạng thái ở máy người chơi client.
- **Ưu điểm:**
  - + Mô hình đơn giản giúp dễ triển khai, phát triển nhanh, phù hợp với game nhỏ.
  - + Không tốn chi phí thuê máy chủ riêng.
- **Nhược điểm:**
  - + Hệ thống phụ thuộc vào cấu hình cũng như mạng của máy Host, máy Host lag thì tất cả người chơi khác đều ảnh hưởng và khi máy Host sập thì game cũng sập.
  - + Tính bảo mật, chống gian lận kém vì người chơi Host quản lý toàn bộ thông tin tất cả người chơi trong phòng và trạng thái gameplay.

4.3.2 Mô hình Client-Server (Dedicated Server)

Với mô hình Client-Server, server được chạy trên máy riêng (thường là máy áo), không phải là người chơi. Các người chơi là client sẽ kết nối với server và gửi dữ liệu hoặc gọi lệnh từ server, server sẽ xử lý logic và trả về cho client.

- **Đặc điểm chính:**
  - + Server là máy chạy độc lập (24/7)
  - + Logic gameplay chỉ được chạy trên server
  - + Client chỉ đóng vai trò hiển thị dữ liệu được server xử lý.



Hình 14: Mô hình Client-Server (Dedicated Server)

- **Ưu điểm:**
  - + Tính ổn định cao vì server là máy chạy độc lập, không bị ảnh hưởng bởi người chơi. Người chơi rốt mạng hoặc thoát thì server vẫn chạy.
  - + Tính công bằng, chống gian lận cao do mọi logic và dữ liệu gameplay tập trung ở server, người chơi không có quyền xem dữ liệu người chơi khác cũng như chỉnh sửa dữ liệu gameplay.
  - + Khả năng mở rộng cao khi cần nâng cấp hệ thống (tăng tốc độ truyền, tăng lượng người chơi đồng thời) vì chỉ cần nâng cấp cấu hình máy ảo.
- **Nhược điểm:**
  - + Tốn chi phí vận hành máy chạy server.
  - + Độ phức tạp cao hơn vì cần có kiến thức về hosting, devops và tối ưu cấu trúc code cho headless server.

4.3.3 Lý do chọn kiến trúc Client-Host ở giai đoạn đầu

Nhóm quyết định chọn kiến trúc Client-Host cho giai đoạn đầu của dự án vì 2 lý do chính là tối ưu tốc độ phát triển dự án và tối ưu về chi phí.

- Tối ưu tốc độ phát triển dự án:
  - + Kiến trúc Client-Host có thể triển khai dễ dàng trong Unity Editor, không cần triển khai backend, nghiên cứu hosting/devops.
  - + Nhóm phát triển có thể tập trung hơn vào phát triển gameplay, UI của game.

+ Vì người chơi Host cũng chính là Server nên mỗi lần chơi thử không cần tốn thời gian theo dõi trạng thái server, thuận thiện hơn cho quá trình tìm lỗi.
+ Giai đoạn phát triển chưa cần quá quan tâm đến độ trễ (không quá ảnh hưởng đến game dạng turn-based) cũng như tính công bằng, bảo mật.

- Tối ưu chi phí: Dự án giai đoạn thử nghiệm ban đầu chưa cần phải thuê dịch vụ máy ảo/hosting nên không tốn chi phí.

4.3.4 Lý do chuyển sang kiến trúc Client-Server ở giai đoạn sau

Nhóm dự kiến sẽ chuyển kiến trúc mạng từ Client-Host sang Client-Server trong giai đoạn sau của dự án, sau khi đã hoàn thiện các logic gameplay cốt lõi và cải tiến UI của game. Mục đính chỉnh của việc này là nhằm tăng chất lượng game về nhiều mặt:

- Tính ổn định: Server không còn phụ thuộc vào máy người chơi, điều này giúp cho người chơi có thể rời phòng bất ký lúc nào (kể cả host) mà không làm gián đoạn game.
- Tính công bằng, bảo mật: Chỉ có server mới giữ và xử lý toàn bộ thông tin người chơi và gameplay, người chơi chỉ có quyền xem dữ liệu của bản thân.
- Tính mở rộng: Việc xây dựng game trên kiến trúc Client-Server giúp cho nhóm phát triển có thể dễ dàng mở rộng hệ thống game hơn vì khi server không còn là người chơi, việc mở rộng server sẽ trở nên dễ dàng hơn. Các nền tảng phát hành game cũng đòi hỏi điều này đối với các tựa game multiplayer.

4.4 Giải pháp voice chat Vivox

Trong các game multiplayer, đặc biệt là dạng board game như nhóm đang triển khai, hệ thống voice chat đóng vai trò rất quan trọng trong việc giúp tăng độ tương tác giữa các người chơi.

Hiện tại trên thị trường có một số giải pháp voice chat có thể tich hợp vào Unity như Vivox, Photon Voice 2, Dissonance Voice và Steam Voice Chat. Sau đây nhóm sẽ đưa ra lý do giải pháp Vivox được chọn để triển khai bằng cách đánh giá tổng quan các giải pháp trên.

- **Vivox:**
  - Miễn phí lên đến tối đa 5000 người dùng hoạt động cùng lúc (5000 PCU)
  - Được tích hợp trực tiếp trong Unity Service.
  - API gọn, hỗ trợ channel-based, tùy chỉnh âm lượng theo người chơi.
  - Chất lượng âm tốt, phù hợp, ổn định với game giải trí.
- **Photon Voice 2:**
  - Miễn phí tới 20 CCU (kèm theo của dịch vụ Photon Cloud).

- Phù hợp với game dạng realtime (FPS, action).
- Overkill với dạng game turn-based, ít hành động.

- **Dissonance Voice Chat:**
  - Giá công cụ 50 USD (Unity Asset Store).
  - Chạy local server, tính tùy biến cao.
  - Phải tự thiết lập backend hoặc tích hợp thủ công với NGO.
  - Tốt cho dạng game LAN.
- **Steam Voice Chat:**
  - Miễn phí và ổn định.
  - Chỉ dùng được cho game phát hành trên Steam.
  - Không hỗ trợ chia channel dễ dàng như Vivox.

Từ các đặc điểm trên, Vivox là giải pháp tốt nhất không chỉ về chi phí, tính năng mà còn về phù hợp nhất với thể loại game và phù hợp cả với hệ thông NGO mà nhóm đã chọn trước đó. Do đó, nhóm chọn sử dụng Vivox để phát triển hệ thống voice chat trong game và có thể mở rộng qua hệ thống text chat trong tương lai.

4.5 Design Pattern trong dự án

4.5.1 Singleton

Singleton là một trong những mẫu thiết kế phổ biến nhất trong phát triển game, đặc biệt trong Unity. Mẫu này đảm bảo chỉ tồn tại duy nhất một thể hiện (*instance*) của một lớp trong suốt vòng đời ứng dụng, đồng thời cung cấp điểm truy cập toàn cục tới thể hiện đó. Singleton thường được sử dụng cho các hệ thống quan trọng như `GameManager`, `AudioManager`, `UIManager` hoặc `NetworkManager`.

- **Ưu điểm:**
  + **Dễ truy cập và tiện lợi:** Mọi thành phần trong game có thể dễ dàng lấy instance thông qua thuộc tính tĩnh, ví dụ `GameManager.Instance`, giảm nhu cầu truyền tham chiếu.
  + **Đảm bảo tính duy nhất:** Ngăn chặn việc tạo nhiều đối tượng quản lý gây xung đột tài nguyên hoặc trạng thái.
  + **Phù hợp cho các hệ thống global:** Các hệ thống cần tồn tại xuyên suốt nhiều scene có thể hoạt động ổn định nhờ `DontDestroyOnLoad`.
  + **Dễ triển khai:** Chỉ cần cấu trúc đơn giản, thuận lợi cho cả lập trình viên mới.

- **Nhược điểm:**
  + **Tạo ra Global State:** Việc quản lý trạng thái toàn cục khiến quá trình debug khó khăn hơn và dễ phát sinh lỗi không lường trước.
  + **Tăng sự phụ thuộc (tight coupling):** Các lớp trong hệ thống dễ bị phụ thuộc trực tiếp vào Singleton, dẫn đến kiến trúc kém linh hoạt.
  + **Khó khăn trong Unit Testing:** Singleton sử dụng trạng thái tĩnh, khiến việc reset hoặc mock dữ liệu trở nên phức tạp.
  + **Dễ bị lạm dụng:** Việc dùng Singleton không hợp lý có thể phá vỡ tính module hóa và vi phạm nguyên tắc thiết kế SOLID.
  + **Vấn đề cạnh tranh tài nguyên (concurrency):** Trong môi trường multiplayer hoặc multithread, Singleton cần cơ chế đồng bộ để tránh lỗi race condition.

4.5.2 Observer Pattern

Observer Pattern là một mẫu thiết kế hành vi cho phép nhiều đối tượng (*observers*) đăng ký và theo dõi sự thay đổi từ một nguồn phát (*subject*). Khi trạng thái của *subject* thay đổi, tất cả các *observers* sẽ tự động được thông báo và cập nhật theo. Phương pháp này giúp loại bỏ sự phụ thuộc chặt chẽ giữa các đối tượng và mang lại khả năng mở rộng linh hoạt, đặc biệt trong những hệ thống có nhiều thành phần cần phản ứng đồng bộ với dữ liệu hoặc sự kiện.

Trong phát triển game bằng Unity, Observer Pattern trở nên đặc biệt hiệu quả nhờ kiến trúc dựa trên `ScriptableObject` theo khuyến nghị trong bài trình bày *"Game Architecture with Scriptable Objects"* tại Unite Austin 2017 [5]. Thay vì để các GameObject gọi trực tiếp lẫn nhau hoặc tạo chuỗi tham chiếu phức tạp, Unity có thể sử dụng các *Event Channels*—một dạng ScriptableObject đóng vai trò trung gian. Các hệ thống khác nhau (UI, gameplay, NPC, player controller, animation, v.v.) chỉ cần đăng ký lắng nghe các sự kiện được phát ra từ một Event Channel, mà không cần biết đối tượng nào đã gửi sự kiện.

Cách tiếp cận này mang lại nhiều ưu điểm:

- **Giảm phụ thuộc (decoupling)** giữa các hệ thống: sender và receiver không cần tham chiếu trực tiếp đến nhau.
- **Dễ mở rộng và bảo trì**: thêm hoặc bỏ một observer không ảnh hưởng luồng xử lý.
- **Tái sử dụng logic**: cùng một sự kiện có thể được nhiều hệ thống lắng nghe, như UI cập nhật thông tin, hiệu ứng âm thanh, hoặc logic gameplay.
- **Thân thiện với workflow của Unity**: ScriptableObject sống độc lập với scene, giúp các sự kiện có thể dùng chung xuyên suốt toàn bộ trò chơi.

Với dự án, Observer Pattern được ứng dụng vào nhiều cơ chế như: truyền sự kiện thay đổi trạng thái người chơi, thông báo khi người chơi tương tác đồ vật, cập nhật UI theo hành

động trong Cabin, hay đồng bộ các sự kiện bắt đầu/thoát khỏi bàn board game. Nhờ sử dụng Event Channels, các thành phần trong game chỉ cần đăng ký lắng nghe sự kiện (OnGameStart, OnSitDown, OnInteractObject, ...) mà không cần duy trì quan hệ tham chiếu phức tạp giữa các script.

Áp dụng Observer Pattern theo phương pháp từ Unite Austin 2017 giúp kiến trúc của trò chơi gọn gàng hơn, dễ mở rộng và ít lỗi hơn, đồng thời phù hợp với quy mô một dự án nhiều người chơi, nơi các hệ thống UI, network, hành vi nhân vật và bàn board game cần được đồng bộ và hoạt động trơn tru.

Listing 1: Event Channel trong dự án

```
using UnityEngine;
using UnityEngine.Events;

/// <summary>
/// This class is used for Events that have no arguments (Example: Start game
    event)
/// </summary>
[CreateAssetMenu(menuName = "Events/Void Event Channel")]
public class VoidEventChannelSO : ScriptableObject
{
    public UnityAction OnEventRaised = delegate { };

    public void RaiseEvent() => OnEventRaised.Invoke();
}
```

4.5.3 State Pattern

State Pattern là một mẫu thiết kế quan trọng trong phát triển game hiện đại, được trình bày rõ trong cuốn *Game Programming Algorithms and Techniques*[2]. Mục tiêu của mẫu thiết kế này là quản lý hành vi của một thực thể thông qua nhiều trạng thái khác nhau mà không cần sử dụng các cấu trúc điều kiện phức tạp như `if-else` hoặc `switch-case`.

Thay vào đó, mỗi trạng thái được tách thành một lớp độc lập, chịu trách nhiệm cho ba giai đoạn vòng đời chính: **OnEnter**, **OnTick** và **OnExit**. Cách tiếp cận này mang lại nhiều lợi ích: mã nguồn rõ ràng và dễ bảo trì, hành vi được tách bạch theo trạng thái, dễ mở rộng thêm trạng thái mới và dễ dàng kiểm soát quá trình chuyển đổi giữa các trạng thái.

Trong dự án, người chơi có nhiều trạng thái khác nhau như *Idle*, *Move*, *Jump*, *Sit*, v.v. Mỗi trạng thái được xây dựng dựa trên interface `IState` để đảm bảo cấu trúc nhất quán và dễ dàng quản lý.

Listing 2: Interface IState trong dự án

```
public interface IState
{
    public void OnEnter();
    public void OnTick();
    public void OnExit();
}
```

Interface trên định nghĩa một quy tắc chung cho tất cả các trạng thái. Mọi trạng thái mới đều phải thực hiện ba phương thức này, từ đó đảm bảo rằng vòng đời của trạng thái được tuân thủ chặt chẽ.

Listing 3: Ví dụ: IdleState

```
public class IdleState : IState
{
    protected Animator _animator;
    protected PlayerController _host;
    protected static readonly int _animHash = Animator.StringToHash("Idle");

    public IdleState(PlayerController host, Animator animator)
    {
        _host = host;
        _animator = animator;
    }

    public virtual void OnEnter()
    {
        _animator.Play(_animHash);
        _host.Movement = Vector3.zero;
    }

    public virtual void OnExit() { }

    public virtual void OnTick() { }
}
```

Ví dụ trên mô tả trạng thái *Idle*. Khi bước vào trạng thái này, nhân vật sẽ phát animation đứng yên và đặt vector chuyển động về 0. Trạng thái tự quản lý hành vi của nó, không gây ảnh hưởng đến các trạng thái khác, đúng theo tinh thần của State Pattern.

Listing 4: Hàm chuyển trạng thái trong State Machine

```
private void ToState(IState newState)
{
    if (_currentState == newState) return;
    _currentState.OnExit();
```

```
    _currentState = newState;
    _currentState.OnEnter();
}
```

Hàm `ToState` là trung tâm của hệ thống State Machine: đảm bảo quá trình thoát khỏi trạng thái cũ và khởi tạo trạng thái mới được thực hiện đúng thứ tự. Đồng thời, việc gọi `OnTick` mỗi khung hình sẽ giúp trạng thái hiện tại xử lý logic của nó một cách độc lập.

Listing 5: Hàm Update trong game loop

```
void Update()
{
    if (!IsOwner)
        return;
    _currentState.OnTick();
}
```

Tổng kết lại, việc áp dụng State Pattern giúp hệ thống điều khiển nhân vật trong dự án trở nên linh hoạt, dễ mở rộng, rõ ràng và ít lỗi hơn, phù hợp với kiến trúc được khuyến nghị trong tài liệu của Sanjay Madhav.

5 Thiết kế hệ thống

5.1 Kiến trúc tổng thể

Dự án được xây dựng bằng Unity, sử dụng công cụ NGO (Network for GameObjects) cho phần Networking (multiplayer) và thiết kế dựa trên mô hình Client-Host ở giai đoạn này (giai đoạn 1).

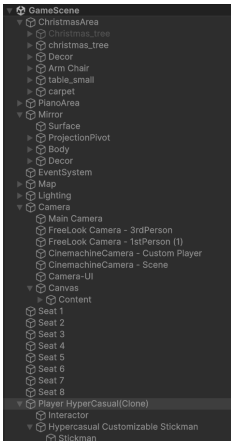
5.1.1 Kiến trúc dự án Unity

Một dự án được xây dựng trên kiến trúc gồm có 3 lớp chính:

- Lớp Asset: chứa các model 3D, texture, material, file nhạc, âm thanh, script, prefab,...
- Lớp Runtime: gồm các Scene và toàn bộ GameObject cùng với các component của nó khi game chạy.
- Lớp Engine System: gồm hệ thống vật lý, âm thanh, render, input, event loop của engine Unity.

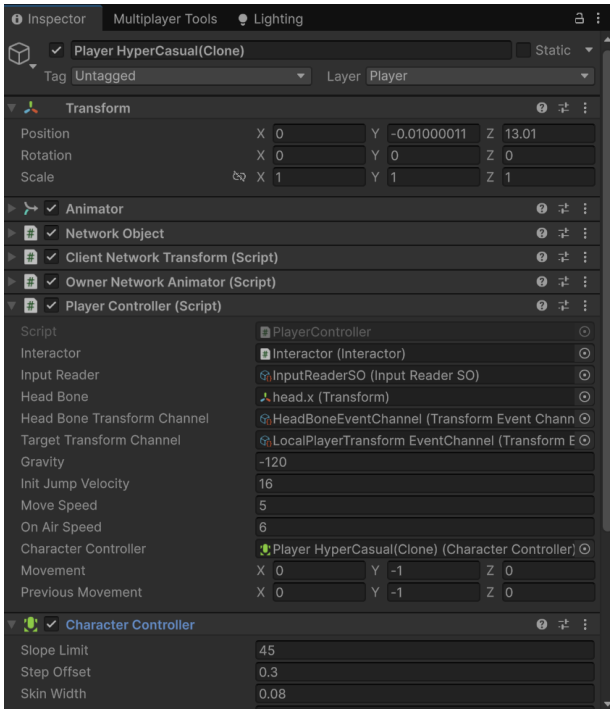
Các lớp được hoạt động theo mô hình pipeline của Unity. Trong quá trình phát triển, lập trình viên tương tác chính với các GameObject, Component và các tài nguyên của game (lớp Asset). Sau đây nhóm sẽ nêu các đặc điểm chính của các thành phần quan trọng mà nhà phát triển sẽ làm việc trong dự án Unity:

- **GameObject**:
  - Đây là thực thể cơ bản nhất trong Scene, nó hoạt động như một container trống chứa các Component.
  - Đại diện cho các đối tượng trong game như: nhân vật, camera, UI, map,...
  - Không chứ bất kì logic nào, tất cả logic, hành vi do các component xử lý.
  - Tổ chức cấu trúc dạng cha-con (hierachy) trực quan.



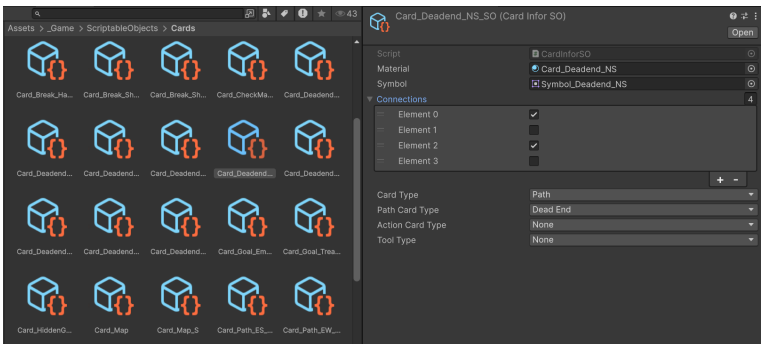
Hình 15: Cấu trúc mẫu của các GameObject trong Scene (GameScene)

- **Component**:
  - Là nơi định nghĩa các hành vi và dữ liệu của GameObject. Ví dụ như Component Transform là một component bắt buộc có của mọi GameObject, nó giúp biểu diễn vị trí (tọa độ, hướng) và scale của đối tượng.
  - Gồm các Component được viết sẵn của Unity và các script do lập trình tự viết.
  - Tính tái sử dụng cao, dễ mở rộng.
- **MonoBehaviour**:
  - Là một class nền tảng của Unity Engine, cung cấp các callback trong lifecycle của Unity:
    + Awake(): khởi tạo dữ liệu của class, chạy khi object được load vào Scene.
    + Start(): chạy duy nhất 1 lần ngay trước frame đầu tiên sau khi object được kích hoạt.
    + Update(): chạy mỗi frame khi được kích hoạt.
    + FixUpdate(): tương tự Update nhưng FPS được giữ cố định, thường dùng cho các logic vật lý.
    + LateUpdate(): chạy ngay sau Update chạy xong.
    + OnEnable()/OnDisable(): chạy khi đối tượng được bật/tắt.
    + OnDestroy(): Chạy khi object bị xóa khỏi Scene, dùng để dọn dẹp tài nguyên.
  - Hỗ trợ liên kết dữ liệu giữa các component với hệ thống của Unity, hỗ trợ lập trình event-driven.



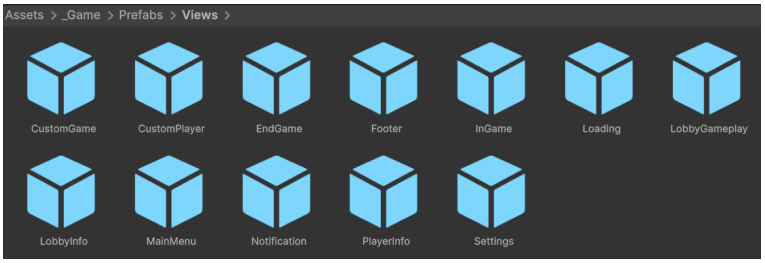
Hình 16: Cấu trúc mẫu của các Component trong GameObject (Player)

- **ScriptableObject**:
  - Là một asset chứa dữ liệu, độc lập với scene và lifetime của Unity.
  - Thường dùng lưu trữ dữ liệu tĩnh, có thể tái sử dụng cho nhiều đối tượng như Stats nhân vật, Skill data, Game Config...
  - Phù hợp cho mô hình Data-Oriented.



Hình 17: Các scriptable object sử dụng để lưu dữ liệu Card trong game

- **Prefab**:
  - Là một asset lưu trữ toàn bộ thông tin của một GameObject.
  - Dùng để tái sử dụng các đối tượng trong game như nhân vật, card ...
  - Có cơ chế Prefab Variant giúp tạo các Prefab con kế thừa Prefab gốc (Prefab cha thay đổi thì Prefab con cũng thay đổi theo).



Hình 18: Các prefab lưu trữ GameObject của UI các screen trong game

- **Scene**:
  - Là không gian chứa tất cả các GameObject.
  - Thường được chia ra theo module để dễ quản lý: Boot Scene, Gameplay...
  - Có thể mở nhiều scene cùng lúc (thường dùng cho tách riêng scene UI và scene gameplay).

5.1.2 Kiến trúc NGO

NGO là công cụ hỗ trợ xây dựng game multiplayer dựa theo mô hình GameObject do Unity phát triển. NGO có mục tiêu chính là đơn giản hóa các đồng bộ trạng thái, xử lý RPC (remote procedure call), quản lý các kết nối người dùng trong mạng, đồng thời hỗ trợ các kiến trúc mạng khác nhau như client-host và client-server authoritative,

Trong NGO, vòng đời mạng tổng quan như sau:

+ Khởi tạo **NetworkManager**.

+ Chọn chế độ chạy: Host/Client/Server.

+ Server khởi tạo danh sách các kết nối.

+ Server tạo các NetworkObject và thông báo tới Client.

+ Client tạo bản sao và đồng bộ các trạng thái.

+ Vòng lặp cập nhật (network tick), xử lý RPC.

+ Dọn dẹp khi kết thúc: hủy các NetworkObject và hủy các kết nối mạng.

Sau đây, nhóm sẽ giới thiệu các thành phần chính được sử dụng trong một dự án dùng NGO:

- **NetworkManager**:
  - Là thành phần (Component) điều khiển chính của hệ thống mạng, quản lý đồng thời nhiều chức năng:
    + Khởi tạo server, host hoặc client.
    + Điều khiển tầng vận chuyển (Unity Transport/Custom Transport)
    + Kiểm soát cập nhật mạng (tick mạng).
    + Quản lý lifecycle của các NetworkObject trong mạng (tạo/hủy)
    + Lưu trữ trạng thái, thông tin của các đối tượng kết nối trong mạng (client, server).
  - Giao thức mặc định sử dụng là Unity Transport (UTP) hỗ trợ UDP tin cậy, QoS, kết hợp với Relay thông qua Unity Service,
- **NetworkObject**:
  - Là Component bắt buộc cho bất ký GameObject nào muốn tồn tại trong mạng multiplayer, có vai trò quan trọng:
    + Cung cấp mã định danh duy nhất (NetworkInstanceId) giữa tất cả các client.
    + Quy định ai là chủ sở hữu (owner) của object.
  - Chịu sử quản lý của NetworkManager khi tạo/hủy. Các thao tác server thực hiện để tạo (spawn) một NetworkGameobject:
    + Server tạo instance trong scene.
    + Object sẽ được cấp mã định danh.
    + Server gửi thông báo đến tất cả các client.
    + Client tạo bản sao tương ứng với NetworkObject được tạo trên server,
- **NetworkBehaviour**:
  - Là một Component kế thừa từ MonoBehaviour và có thêm thêm các tính năng về mạng:
    + Hỗ trợ RPC.
    + Có thể sở hữu được NetworkVariable.
    + Hỗ trợ các callback mạng như: OnNetworkSpawn(), OnNetworkDespawn(),...
  - Được sử dụng đồng thời với MonoBehaviour để tách riêng logic runtime và các logic mạng:
    + MonoBehaviour: xử lý input, animation, các UI local, audio...
    + NetworkBehaviour: Xử lý đồng bộ, RPC, kiểm tra quyền,...
- **RPC (Remote Procedure Call)**:

  Là cơ chế gửi lời gọi hàm giữa client và server. Trong NGO cung cấp 2 loại RPC chính là ServerRpc và ClientRpc.

- **ServerRpc:**
  Gửi từ Client đến Server
  + Đặc điểm: Chỉ chạy trên server, có thể được gửi từ owner hoặc non-owner, có hỗ trợ đảm bảo thứ tự và tin cậy.
  + Ứng dụng: client gửi input cho server, gửi yêu cầu spawn object cho server (ví dụ như bán đạn), thông báo hành động (ví dụ như chọn bài).
- **ClientRpc:**
  Gửi từ Server đến Client
  + Đặc điểm: Gửi đến mọi client hoặc một client cụ thể, thường dùng cho các sự kiện tức thời.
  + Ứng dụng: Thông báo hiệu ứng (ví dụ như nổ, va chạm), cập nhật trạng thái (ví dụ như bắt đầu game).

- **NetworkVariable**:
  - Là cơ chế giúp tối ưu quá trình đồng bộ dữ liệu game, tự động cập nhật trạng thái thay vì phải gửi RPC thủ công (giảm tải RPC).
  - Hỗ trợ phân quyền đọc/ghi rõ ràng.
  - Hỗ trợ callback khi giá trị thay đổi.
  - Ứng dụng phổ biến trong đồng bộ thời gian (cooldown/countdown), điểm, máu...
- **NetworkList**:
  - Là một dạng NetworkVariable chứa danh sách.
  - Tự động đồng bộ các thay đổi trên danh sách (thêm, xóa, sửa) mà không cần gửi toàn bộ danh sách.
  - Ứng dụng phổ biến với danh sách người chơi, kho đồ (inventory), bảng điểm,...
- **NetworkTransform**:
  - Là Component chuyên dùng để tự động đồng bộ transform (vị trí, góc xoay, scale) của NetworkObject.
  - Hỗ trợ tùy chỉnh thông số muốn đồng bộ để tối ưu tải.
  - Hỗ trợ cơ chế interpolation giúp chuyển động mượt hơn.

5.2 Hệ thống Lobby

Trong game multiplayer, hệ thống Lobby đóng vai trò quan trọng, cung cấp cho người chơi những chức năng chính như:

+ Tạo phòng.

+ Tham gia phòng.

+ Rời phòng.

+ Mời người chơi khác ra khỏi phòng (dành cho host).

+ Kết nối các người chơi trong phòng qua hệ thống trung gian của Unity (Relay).

+ Đồng bộ thông tin người chơi trong phòng.

Kiến trúc tổng thể của hệ thống Lobby gồm 3 phần chính:

- Unity Lobby Service:

  Là một dịch vụ cung cấp sẵn của Unity, quản lý danh sách người chơi, thông tin phòng và thông tin người chơi. Duy trì kết nối của phòng bằng heartbeat (khi không còn người trong phòng thì phòng sẽ bị hủy).

- Unity Relay Service:

  Là dịch vụ của Unity giúp tạo server trung gian để kết nối các người chơi trong phòng, giúp vượt qua các rào cản về NAT và firewall (NAT Tranversal), tăng tính bảo mật nhờ việc che giấu đi địa chỉ IP của người kết nối trong phòng.

- Lớp quản lý trong game:

  Nhóm thiết kế 2 Component là LobbyManager và RelayManager nhằm phụ trách logic phòng và relay cho game thông qua các dịch vụ đã có sẵn của Unity.

Sau đây, nhóm sẽ trình bày luồng hoạt động của các chức năng chỉnh của hệ thống phòng.

5.2.1 Tạo phòng

- Host nhấn "Create Lobby"

- LobbyService được gọi và tạo một phòng cho host.

- RelayService được gọi và tạo Allocation.

- Thông tin Relay Allocation được cập nhật vào dữ liệu của phòng (Relay Join Code).

- Host gọi NetworkManager và bắt đầu kết nối chế độ host (StartHost()).

- Hệ thống bắt đầu gọi Heartbeat và Polling.

- Người chơi host được chuyển sang scene gameplay.

5.2.2 Tham gia phòng

- Client nhấn "Join" vào phòng (từ danh sách phòng được lấy từ LobbyService).
- LobbyService trả về phòng tương ứng với phòng Client đã nhấn.
- Client tham gia Relay Allocation (bằng Relay Join Code trong dữ liệu phòng).
- Client gọi NetworkManager và bắt đầu kết nối chế độ client (StartClient()).
- Hệ thống gọi Polling để cập nhật dữ liệu Lobby.
- Người chơi client được chuyển qua scene gameplay nhờ đồng bộ với host bằng NetworkManager.

5.2.3 Rời phòng

- Nếu là host thì phòng sẽ bị xóa hoàn toàn, nếu là client thì người chơi bị xóa khỏi phòng (sử dụng LobbyService),
- Dừng tất cả các coroutine về heartbeat và polling.
- Người chơi được chuyển về scene main menu.

5.2.4 Kick khỏi phòng

- Host có thể kick người chơi bằng cách xóa người chơi khỏi phòng (sử dụng LobbyService).
- Người chơi bị kick sẽ cập nhật thông tin phòng thông qua polling và thấy mình không còn trong phòng.
- Dừng tất cả các coroutine về heartbeat và polling.
- Người chơi bị kick được chuyển về scene main menu.

5.2.5 Đồng bộ trạng thái phòng (Polling)

Vì hệ thống phòng không phải là realtime nên người chơi phải liên tục lấy dữ liệu mới (mỗi 5 giây).

- Gọi LobbyService để lấy thông tin phòng bằng mã phòng đã lưu lại từ trước.
- Kiểm tra thay đổi dữ liệu của phòng:
  + Phòng còn không?
  + Người chơi có bị kick không?
  + Có người chơi khác vào hay thoát không?
- Gửi các event tương ứng (OnUpdatedCurrentLobby, OnPlayerJoinedLobby, OnPlayerLeftLobby).

5.2.6 Duy trì phòng bằng Heartbeat

Vì hệ thống Unity Lobby sẽ tự động xóa phòng nếu không có heartbeat trong 30 giây, do đó người chơi host sẽ thực hiện coroutine heartbeat mỗi 15 giây dùng LobbyService gửi heartbeat cho phòng.

5.3 Hệ thống voice chat Vivox tích hợp với Lobby

Hệ thống voice chat Vivox mà nhóm là chọn có giao diện tương tác rất dễ dàng tích hợp vào hệ thống Lobby sẵn có. Sau đây nhóm sẽ mô tả luồng hoạt động của hệ thống voice chat này:

- Khi người chơi vào lobby:
  - Người chơi đăng nhập vào hệ thống Vivox nếu chưa đăng nhập.
  - Người chơi tham gia vào channel voice chat theo mã định danh của lobby.
  - Người chơi bắt đầu có thể trò chuyện với các người chơi khác trong channel.
  - Người chơi có thể tùy chỉnh âm lượng (local) của các người chơi khác trong channel.
- Khi người chơi thoát lobby:
  - Người chơi thoát channel voice chat.
  - Nếu người chơi đăng xuất khỏi vivox nếu là thoát hẳn game.

5.4 Thiết kế Gameplay

Bản đồ Cabin là không gian sinh hoạt chung nơi người chơi có thể di chuyển tự do, tương tác với các đồ vật trong nhà và tham gia vào nhiều trò board game khác nhau. Cabin hoạt động như một sảnh chờ mở rộng kết hợp gameplay tương tác, giúp người chơi giao lưu, thao tác vui nhộn và trải nghiệm các trò chơi dạng board game.

- **Mục tiêu tổng quan:** Người chơi khám phá cabin. Tương tác với các vật thể như nhạc cụ, Tủ đồ, máy phát nhạc, bàn ghế,.. Ngồi vào bàn chơi và tham gia vào các mini board game. Mỗi trò chơi có luật riêng. Hiện tại nhóm chỉ tập trung vào 1 board game là *Fishy Business* lấy cảm hứng từ Saboteur. Cabin đóng vai trò môi trường trung gian nơi người chơi tự do khám phá, giao lưu, tương tác.
- **Cơ chế di chuyển và tương tác:**
  - Người chơi điều khiển nhân vật dạng third-person controller. Có thể nhảy và đi trên cầu thang.
  - Cabin chứa nhiều vật thể có thể tương tác: ghế, sofa, tủ, đồ trang trí, cửa đẩy.
  - Tiếp cận tủ đồ cho phép người chơi thay đổi trang phục, tăng tính cá nhân hóa.

  - Tiếp cận nhạc cụ cho phép người chơi chơi nhạc bằng bàn phím, tăng tính giải trí, giết thời gian chết.
  - Khi tiếp cận một bàn board game, người chơi nhấn tương tác để ngồi vào

- **Hệ thống bàn board game:** Dựa trên luật chơi Saboteur đã nghiên cứu:
  - Sau khi đủ số lượng người chơi ngồi vào bàn, một người bất kỳ nhấn "Start" và bắt đầu game.
  - Tay bài hiện ở dưới màn hình, chọn bài và đặt bài bằng chuột.
  - Có thể quan sát bàn và các người chơi khác. Có cơ chế điều khiển đầu bằng chuột giúp người chơi thể hiện gật đầu hay lắc đầu.
  - Kết thúc trò chơi, hiển thị UI thắng/thua.

- **Giao tiếp trong cabin:**
  - Voice chat
  - Chat trong Cabin (global chat).
  - Biểu cảm nhân vật (emote).

- **Điều kiện hoàn thành:** Cabin không có điều kiện thắng thua chung. Người chơi tự do chọn mini-game yêu thích và hoàn thành các ván chơi nhỏ, sau đó quay lại cabin để chọn bàn khác.

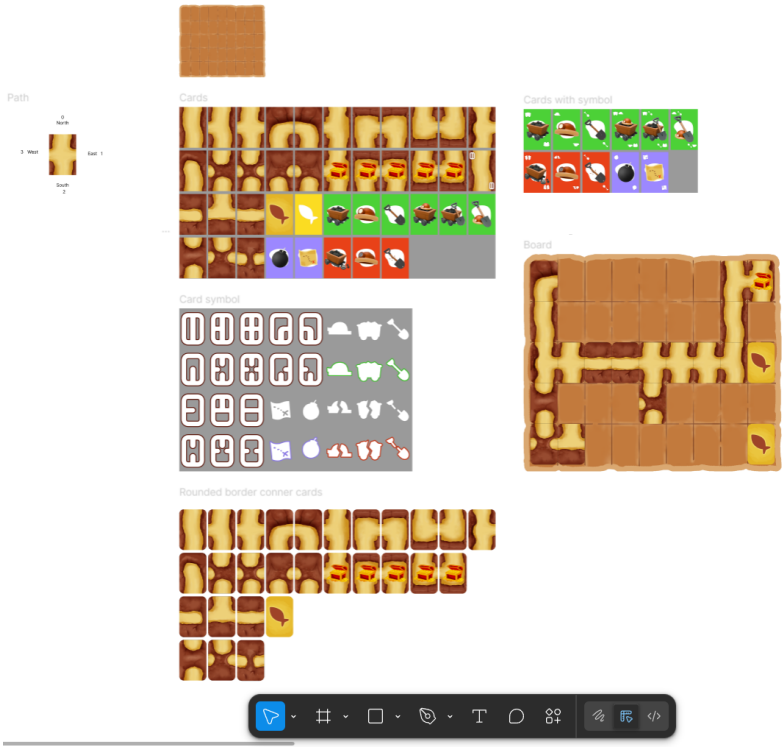
6 Tiến độ dự án

Tuần (Sprint)	Lê Ngọc Hiền	Nguyễn Mạnh Hùng	Huỳnh Trần Học Đăng
Tuần 1–2 (8/9 – 21/9)	• Thiết kế wireframe (Figma) chi tiết • Thiết kế UI tổng thể • Tạo khu vực chờ (lobby) cơ bản	• Tích hợp mô hình Player • Tích hợp mô hình bản đồ • Xây dựng hệ thống camera	• Writeframe Figma (chụp màn hình) • Tạo mô hình Thẻ bài (Card model) • Xây dựng logic thắng/thua
Tuần 3–4 (22/9 – 5/10)	• Tích hợp UI menu chính • Tích hợp phòng tùy chỉnh (custom room) • Tích hợp UI menu khu vực chờ trong game • Thêm cấu hình trò chơi (GameConfig - cấu hình UI)	• Tích hợp mô hình Thẻ bài • Tích hợp mô hình Xu • Xử lý vị trí ngồi • Xử lý vị trí camera	• Triển khai chức năng chia bài • Triển khai chức năng chọn bài • Hoàn thiện logic thắng/thua
Tuần 5–6 (6/10 – 19/10)	• Tích hợp UI Thông tin Lobby • Xây dựng hệ thống hoạt ảnh UI • Cải thiện Lobby (đá người chơi, đấu nhanh - quick match)	• Triển khai chức năng dùng bài • Điều chỉnh camera khi dùng bài • Tích hợp mô hình Thẻ bài và Bàn chơi (Board)	• Triển khai chức năng phát bài (deal card) • Triển khai chức năng rút bài (draw card) • Triển khai chức năng bài chức năng (action card) • Triển khai hệ thống Xu • Triển khai điều kiện thắng
Tuần 7–8 (20/10 – 9/11)	• UI thay đổi thông tin người chơi • UI đổi tên Lobby • Thêm biểu tượng người chơi (icon player) • Hoàn thiện UI kết thúc game	• Triển khai hoạt ảnh di chuyển đầu người chơi • Thêm hoạt ảnh sắp xếp chỗ ngồi • Đồng bộ hóa (Sync) dữ liệu bàn chơi • Sửa lỗi chỗ ngồi và hiển thị	• Sửa lỗi dùng thẻ bài • Sửa lỗi đồng bộ hóa thẻ bài • Thêm đồng hồ đếm ngược theo lượt

Tuần 9–10 (10/11 – 30/11)	• Xây dựng Hệ thống Âm thanh (Audio system) • Thêm hiệu ứng âm thanh (SFX) và nhạc nền • Thêm UI Cài đặt cho âm thanh • Triển khai Trò chuyện bằng giọng nói (Voice chat - Vivox) • Hỗ trợ chơi khác mạng	• Cải thiện ánh sáng trong game scene • Highlight đường đi theo vai trò (role) • Cải thiện di chuyển camera	• Thêm Hiệu ứng hình ảnh (VFX) (thắng/thua, xu, bài chức năng) • Sửa lỗi liên quan tới thẻ bài và hiệu ứng
Tuần 11–12 (1/12 – 14/12)	• Viết báo cáo tổng hợp • Chuẩn bị slide thuyết trình	• Viết báo cáo tổng hợp • Chuẩn bị slide thuyết trình	• Viết báo cáo tổng hợp • Chuẩn bị slide thuyết trình
Tuần 13–14 (15/12 – 28/12)	• Nghiên cứu phát triển giai đoạn 2 • Sửa lỗi tồn đọng	• Nghiên cứu phát triển giai đoạn 2 • Sửa lỗi tồn đọng	• Nghiên cứu phát triển giai đoạn 2 • Sửa lỗi tồn đọng

Bảng 4: Phân công nhiệm vụ giai đoạn 1 theo Sprint

Trải qua quá trình phát triển 14 tuần của giai đoạn 1, dưới đây là 1 số ảnh chụp màn hình gameplay hiện tại của nhóm:



Hình 19: Figma: Bộ thiết kế card



Hình 20: Game Screenshot: Nhân vật khi mới tham gia vào Cabin



Hình 21: Game Screenshot: Người chơi nhảy từ gác xuống tầng 1



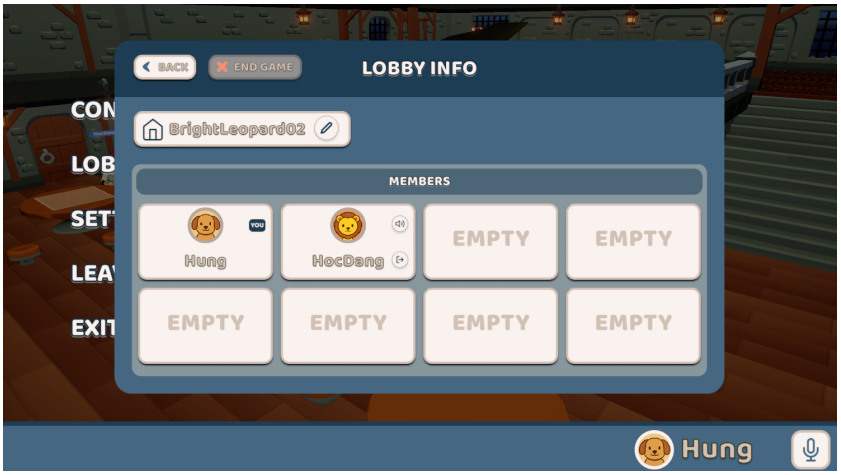
Hình 22: Game Screenshot: Người chơi đứng trước gương thay đổi trang phục



Hình 23: Game Screenshot: Người chơi chơi đàn piano bằng bàn phím



Hình 24: Game Screenshot: Menu trong game



Hình 25: Game Screenshot: Quản lý lobby, có thể đổi tên phòng, kick người chơi,..



Hình 26: Game Screenshot: Tương tác với vật thể khi tới gần



Hình 27: Game Screenshot: Người chơi quay đầu nhìn xung quanh



Hình 28: Game Screenshot: Khung cảnh của 1 bàn chơi dưới góc nhìn mặc định



Hình 29: Game Screenshot: Sử dụng thẻ rockfall (bomb) để phá đường



Hình 30: Game Screenshot: Nối đường tới goal nhưng không chứa vàng



Hình 31: Game Screenshot: Giao diện thắng/thua khi xong game

7 Kế hoạch thực hiện giai đoạn tiếp theo

7.1 Kế hoạch theo mốc thời gian

Giai đoạn tiếp theo được triển khai trong vòng 14 tuần, với mục tiêu mở rộng tính năng, cải thiện trải nghiệm người dùng và nâng cấp kiến trúc hệ thống theo hướng ổn định và dễ mở rộng hơn. Mỗi tuần được phân rã thành một nhóm nhiệm vụ cụ thể, ưu tiên theo mức độ quan trọng và sự phụ thuộc lẫn nhau giữa các tính năng. Kế hoạch chi tiết theo từng mốc thời gian được mô tả trong bảng dưới đây.

Tuần	**Nội dung công việc chi tiết**
1	Xây dựng Text Chat: giao diện chat, danh sách người gửi, chống spam, đảm bảo đồng bộ tin nhắn theo thời gian thực. Cơ chế tính điểm cuối trận và hệ thống emoji.
2	Phát triển hệ thống biểu cảm người chơi (Player's Emotion): tạo thư viện biểu cảm, hoạt ảnh đơn giản, nút chọn nhanh trong giao diện.
3	Cải thiện tương tác với gameplay: thao tác kéo–thả hợp lý hơn, bổ sung hiệu ứng phản hồi trực quan.
4	Thêm các map game mới cho người chơi đa dạng lựa chọn. Tích hợp lựa chọn map lên UI trước khi vào game
5	Tích hợp SFX và VFX: âm thanh thao tác, hiệu ứng mở bài, hiệu ứng hành động, tối ưu hiệu suất tránh gây lag khi kích hoạt liên tục.
6	Chuyển nền tảng sang mô hình Client–Server (nếu tìm được dịch vụ phù hợp): phân tách logic máy chủ, xử lý đồng bộ trạng thái, kiểm thử tải ở quy mô nhỏ.
7-8	Thêm board game mới "Liar Bar!": xây dựng luật chơi cơ bản, UI tối thiểu, chạy thử gameplay loop và kiểm tra lỗi phát sinh.
9	Tối ưu hiệu suất mạng: giảm độ trễ, cải thiện truyền tải sự kiện, xử lý packet loss, kiểm thử với nhiều người chơi đồng thời.
10	Nâng cấp và hoàn thiện UI: thiết kế lại các panel chính, thống nhất style, thêm hiệu ứng chuyển cảnh và icon trực quan hơn.
11	Chuẩn bị bản build phát hành: kiểm thử tính năng, rà soát bug cuối cùng, tối ưu kích thước bản cài.
12	Deploy và phát hành chính thức lên itch.io: tạo trang mô tả, screenshot, trailer ngắn, thiết lập bản public và kênh cập nhật.
13	Thu thập phản hồi từ người chơi: khảo sát, ghi nhận lỗi, đánh giá trải nghiệm, phân loại mức độ ưu tiên của vấn đề.
14	Tổng hợp toàn bộ kết quả, báo cáo tiến độ, đề xuất các cải tiến cho stage tiếp theo và điều chỉnh hướng phát triển lâu dài.

Bảng 5: Bảng kế hoạch thực hiện dự án giai đoạn 2

7.2 Mục tiêu và kết quả mong đợi

- Hoàn thiện đầy đủ các tính năng cốt lõi của Stage 2: giao tiếp, tương tác, bản đồ tùy chỉnh và UI/UX.

- Tối ưu hệ thống mạng để hỗ trợ nhiều người chơi ổn định.
- Ra mắt phiên bản công khai trên itch.io để thu hút người dùng đầu tiên.
- Thu thập dữ liệu phản hồi từ người chơi nhằm cải thiện chất lượng sản phẩm và định hướng tương lai.

8 Tài liệu tham khảo

Tài liệu

[1] AMIGO Spiel + Freizeit GmbH. *Saboteur: Offizielle Spielanleitung*. Regelbuch (Artikel-Nr. 18750), AMIGO Spiel, 2024. Truy cập ngày 13 tháng 12 năm 2025. Available at: https://www.amigo.games/wp-content/uploads/2024/08/18750-ur.pdf

[2] Sanjay Madhav. *Game Programming Algorithms and Techniques: A Platform-Agnostic Approach*. Addison-Wesley, 2013. ISBN: 9780321940155.

[3] Unity Technologies. *Level Up Your Code with Game Programming Patterns*. E-book, Unity Technologies, 2022. Truy cập ngày 13 tháng 12 năm 2025. Available at: https://unity.com/resources/level-up-your-code-with-game-programming-patterns

[4] Unity Technologies. *Create Modular Game Architecture in Unity with ScriptableObjects*. E-book, Unity Technologies, 2023. Truy cập ngày 13 tháng 12 năm 2025. Available at: https://unity.com/resources/create-modular-game-architecture-with-scriptable-objects-ebook

[5] Ryan Hipple. *Unite Austin 2017 - Game Architecture with Scriptable Objects*. Video presentation at Unite Austin 2017, Unity Technologies, 20 tháng 11 năm 2017. Truy cập ngày 13 tháng 12 năm 2025. Available at: https://www.youtube.com/watch?v=raQ3iHhE_Kk

[6] Unity Technologies. *Vivox Unity SDK documentation*. Unity Gaming Services Documentation, phiên bản 16. Truy cập ngày 13 tháng 12 năm 2025. Available at: https://docs.unity.com/ugs/en-us/manual/vivox-unity/manual/Unity/Unity

[7] Unity Technologies. *Netcode for GameObjects Manual (Version 2.7.0)*. Unity Package Documentation. Truy cập ngày 13 tháng 12 năm 2025. Available at: https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.7/manual/index.html

[8] Unity Technologies. *Multiplayer - Unity Manual (Unity 6.2)*. Unity Documentation. Truy cập ngày 13 tháng 12 năm 2025. Available at: https://docs.unity3d.com/6000.2/Documentation/Manual/multiplayer.html

[9] AppWill Inc. *Multiplayer in Unity: Best Networking Solutions 2025 | Ultimate Guide*. Blog post, ngày 24 tháng 6 năm 2025. Truy cập ngày 13 tháng 12 năm 2025. Available at: https://appwill.co/multiplayer-in-unity-best-networking-solutions-2025/